

Del acoplamiento a la comunicación asíncrona

Parte 1 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

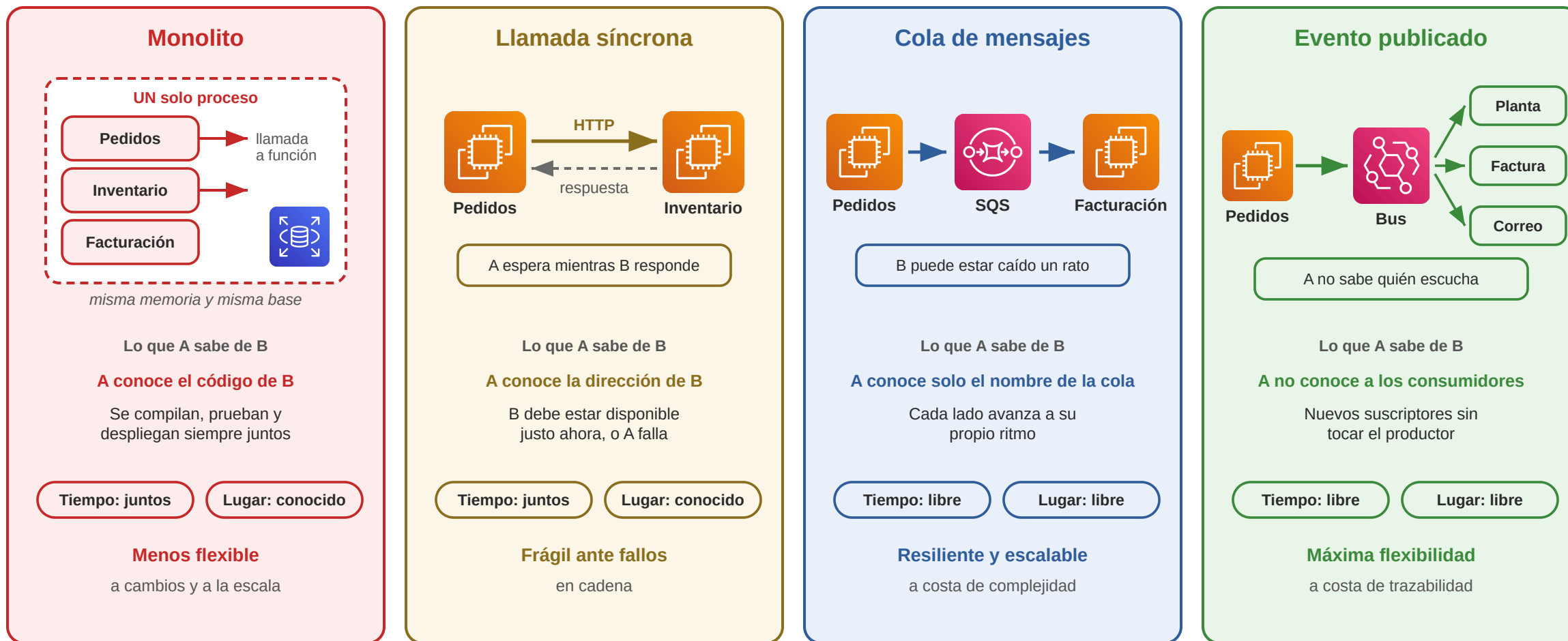
Por qué las llamadas en cadena se rompen y cómo un mensaje intermedio devuelve la resiliencia.

Acoplamiento: cuánto sabe un componente de otro

Cuanto menos necesita saber A sobre B, más libre es cada uno para fallar, escalar y cambiar.

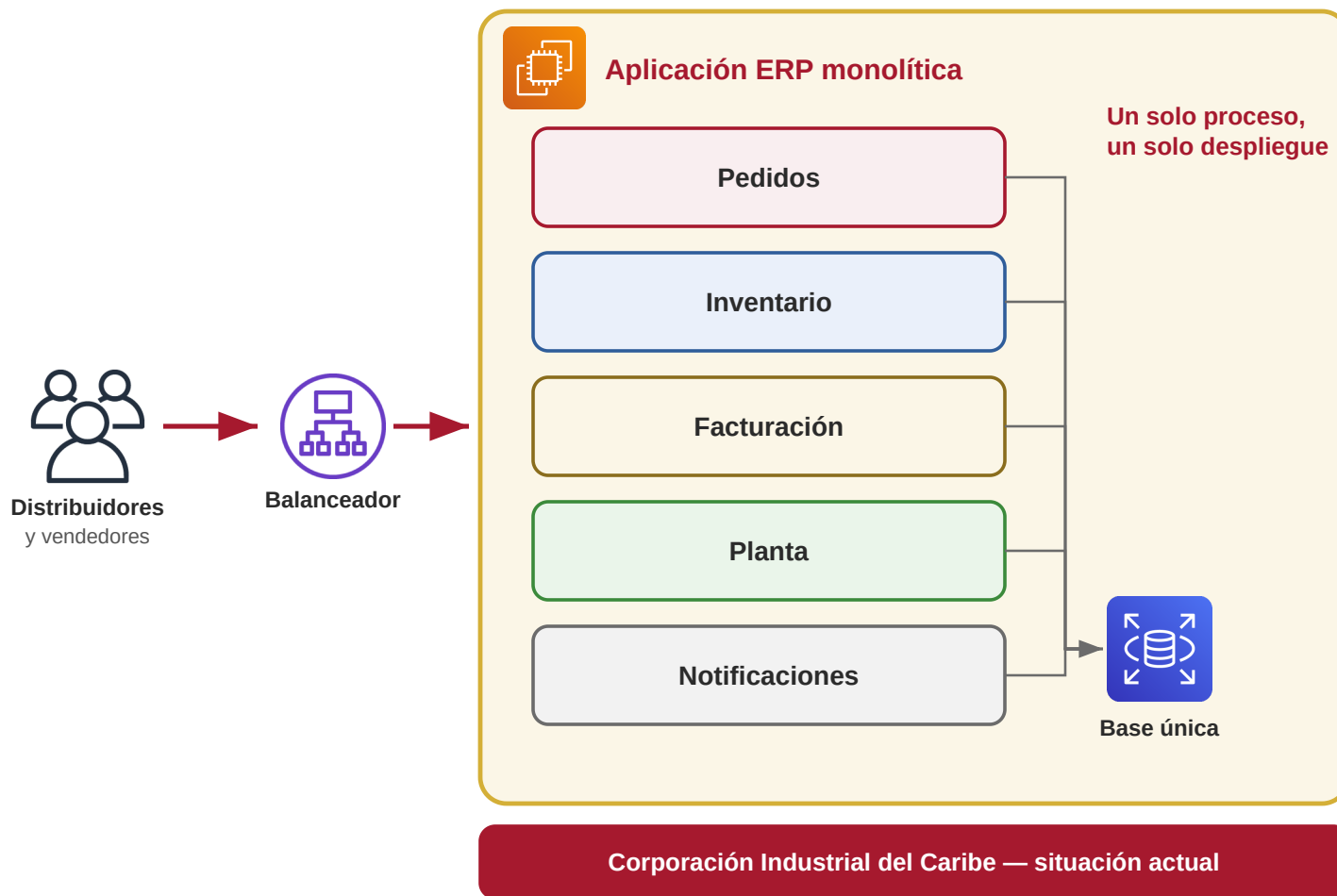
Acoplamiento fuerte

Acoplamiento débil



El monolito de la Corporación: todo en una sola pieza

Un solo proceso y una sola base de datos: simple al inicio, rígido cuando el negocio crece.



1 Despliegue atado

Un cambio pequeño en Facturación obliga a probar y desplegar toda la aplicación

2 Escala atada

Para atender más pedidos hay que clonar también Planta e Inventario

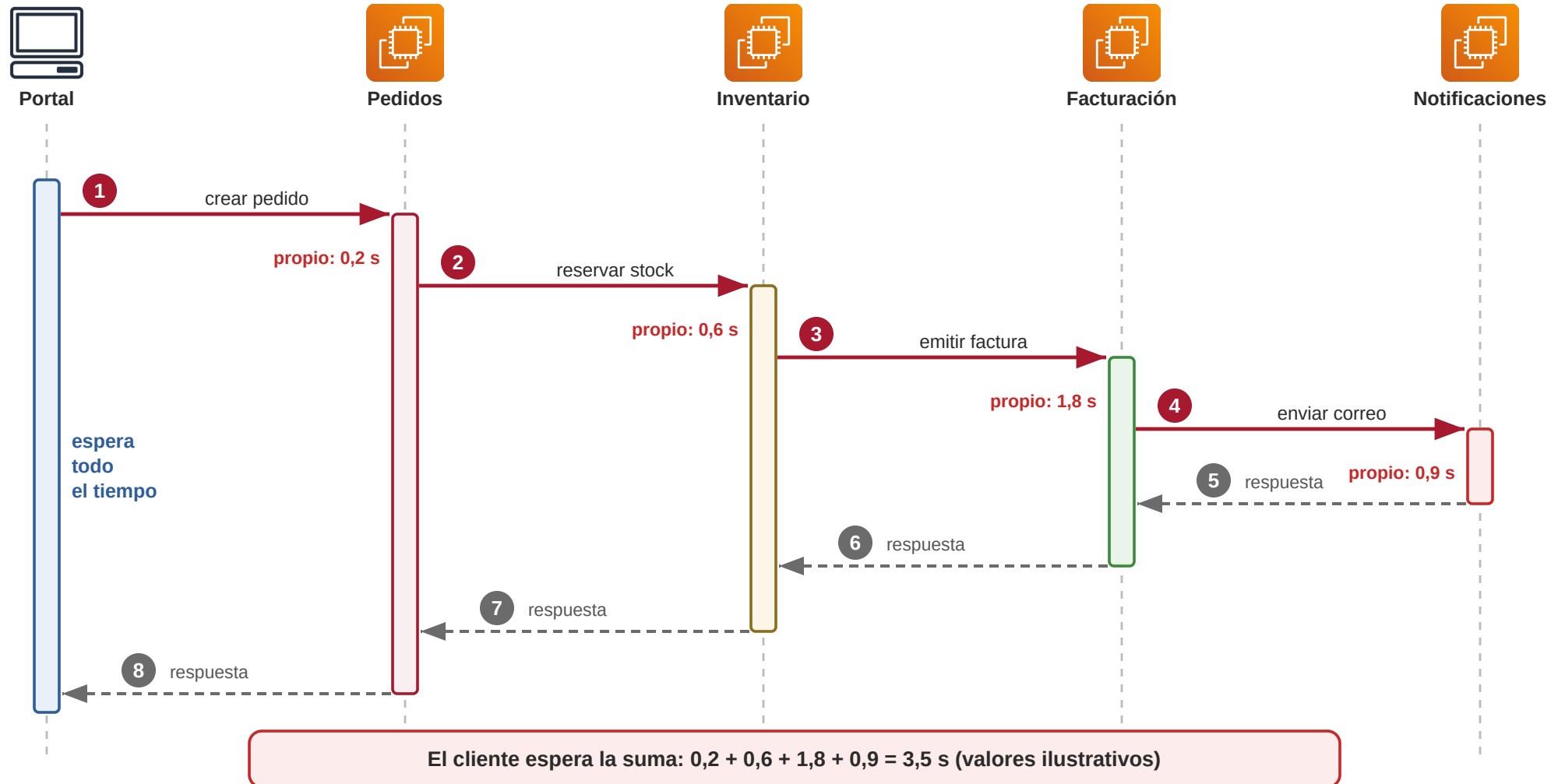
3 Fallos atados

Un error en Notificaciones puede tumbar el proceso completo

Funciona, pero cada cosa depende de todas las demás

Llamadas síncronas en cadena: cada paso espera al siguiente

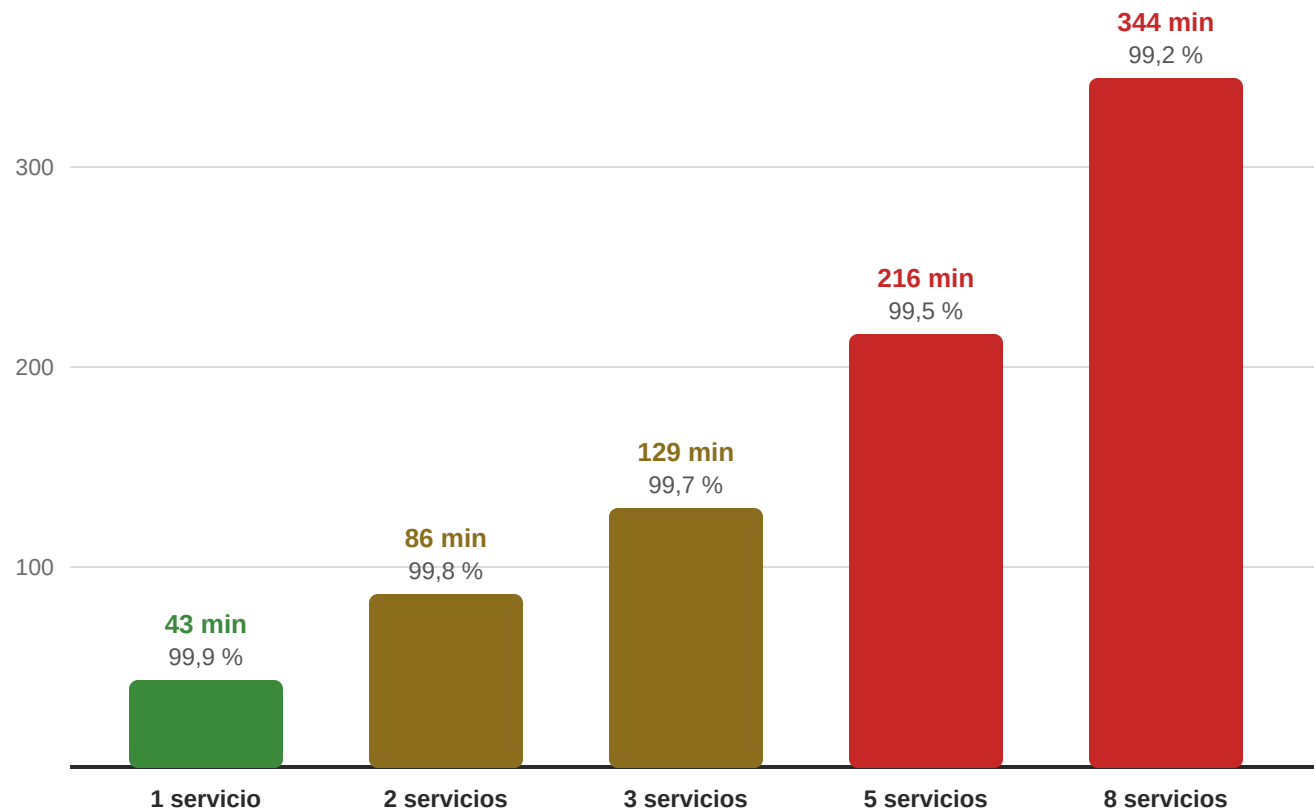
La latencia se suma y el cliente queda atado a la dependencia más lenta de la cadena.



Disponibilidad compuesta: cada eslabón resta

En una cadena síncrona la disponibilidad total es el producto de todas, siempre menor que la peor.

Minutos de caída por mes (30 días)

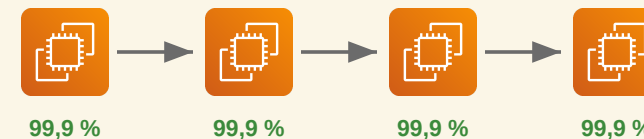


Cada servicio con 99,9 % de disponibilidad, encadenados en serie

La matemática de la cadena

$$\text{Disp. total} = D1 \times D2 \times D3 \times \dots \times Dn$$

Ejemplo con cuatro servicios



$$0,999 \times 0,999 \times 0,999 \times 0,999 = 0,996$$

Disponibilidad efectiva: 99,6 %

Con 8 eslabones: 99,2 %

casi 6 horas de caída al mes

Ningún eslabón es malo por sí solo:
el problema es multiplicarlos en serie.
Desacoplar rompe la serie.

Fallo en cascada: así cae todo por un eslabón lento

La saturación se propaga hacia atrás porque cada servicio mantiene hilos y conexiones esperando.



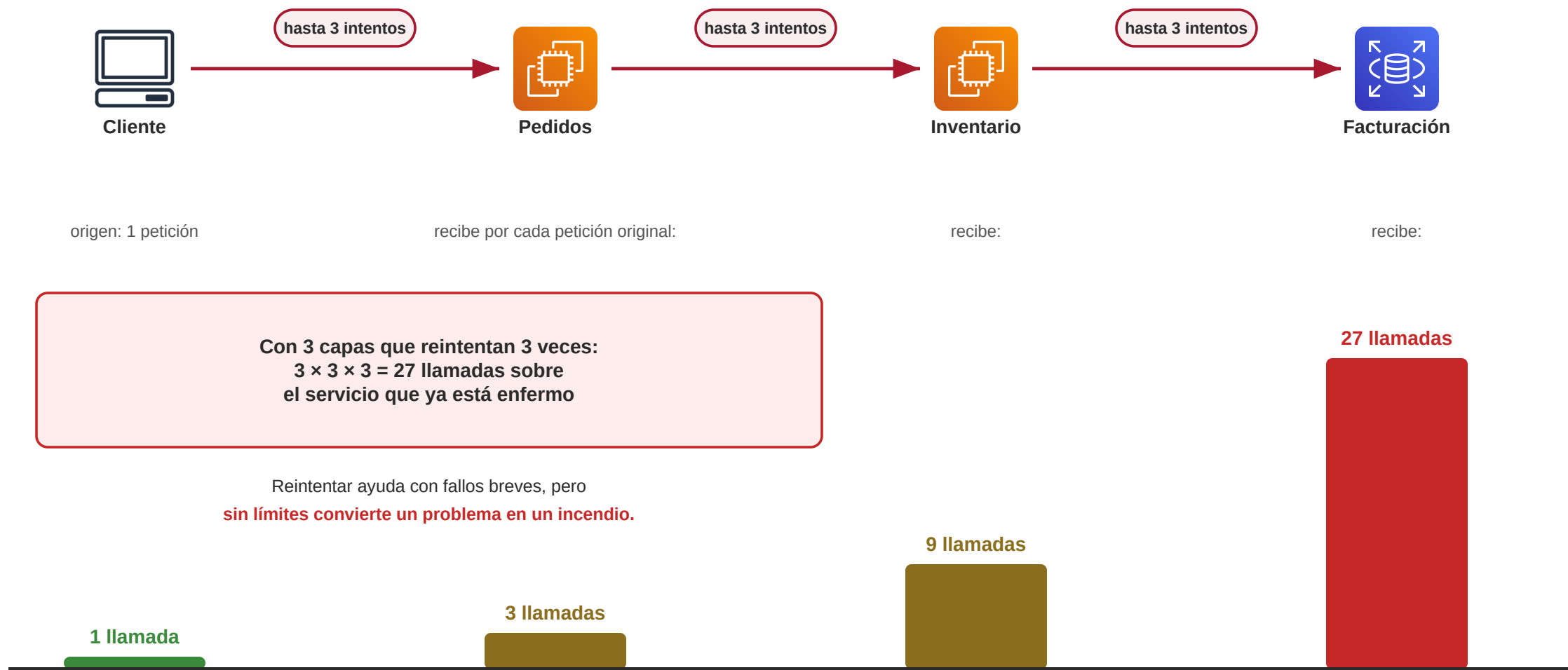
Un componente lento es peor que uno caído: quien falla rápido libera recursos; quien tarda los retiene.

El daño viaja río arriba: de la dependencia más lenta hacia el usuario final.

Tiempos y porcentajes ilustrativos de un incidente típico.

Tormenta de reintentos: el cliente empeora el incendio

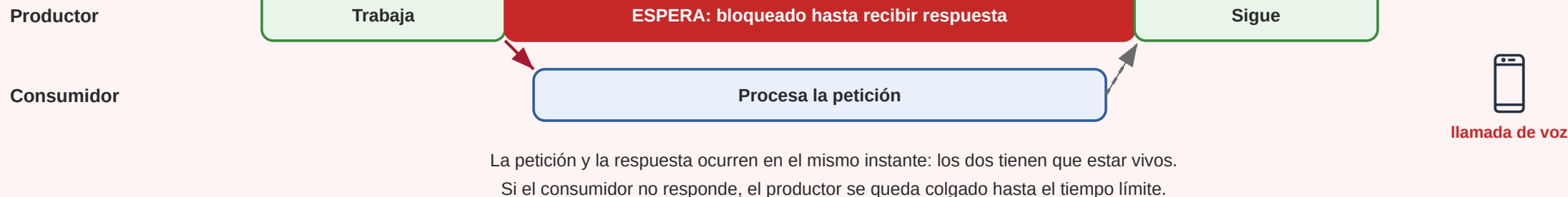
Si cada capa reintenta, el servicio enfermo recibe la carga multiplicada justo cuando menos puede.



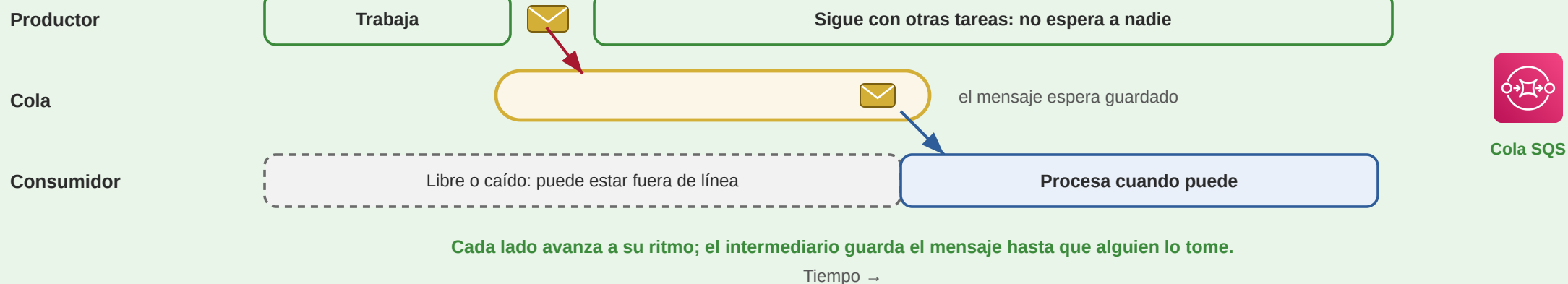
Comunicación asíncrona: dejar el mensaje y seguir

El productor entrega el mensaje a un intermediario y continúa; el consumidor lo toma cuando puede.

Comunicación síncrona: llamar por teléfono y esperar en línea



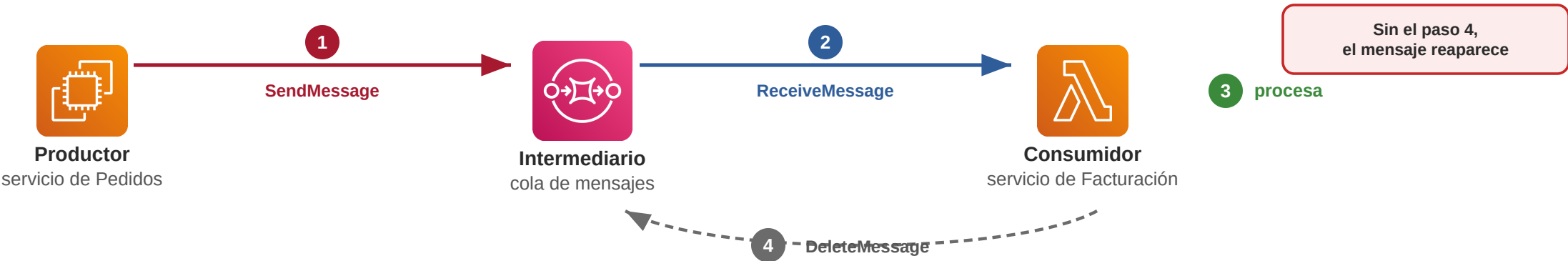
Comunicación asíncrona: mandar un audio por WhatsApp y seguir con tu día



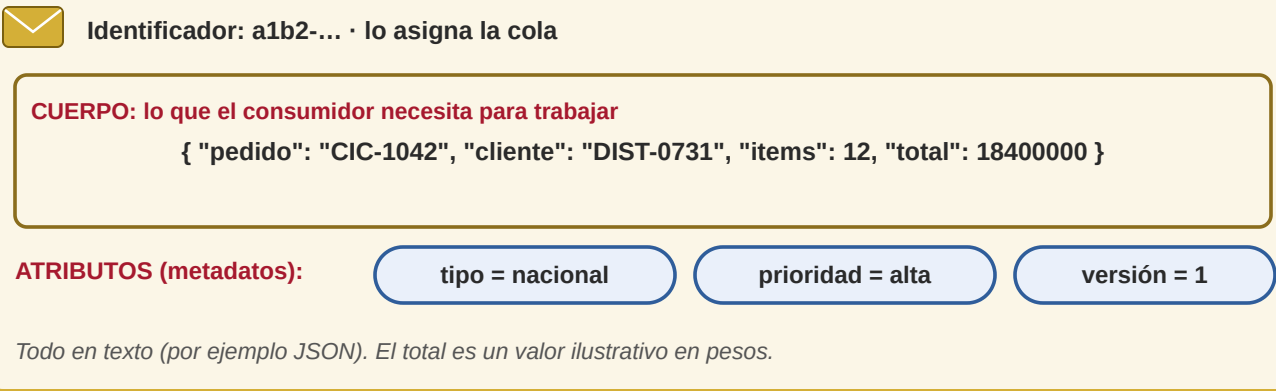
Productor, mensaje, intermediario y consumidor

Cuatro actores y cuatro pasos: enviar, recibir, procesar y confirmar borrando el mensaje.

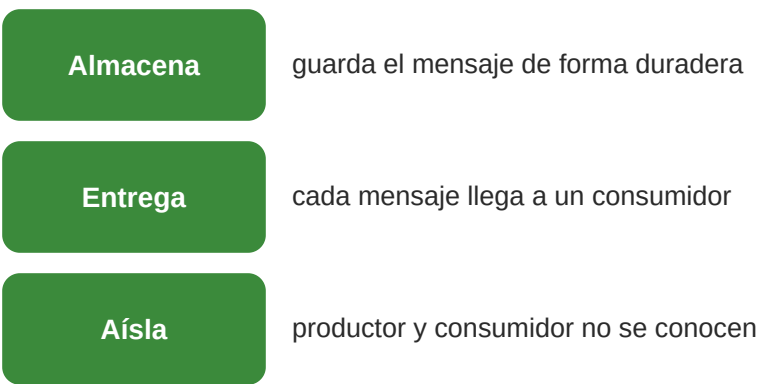
Los cuatro actores



Anatomía de un mensaje



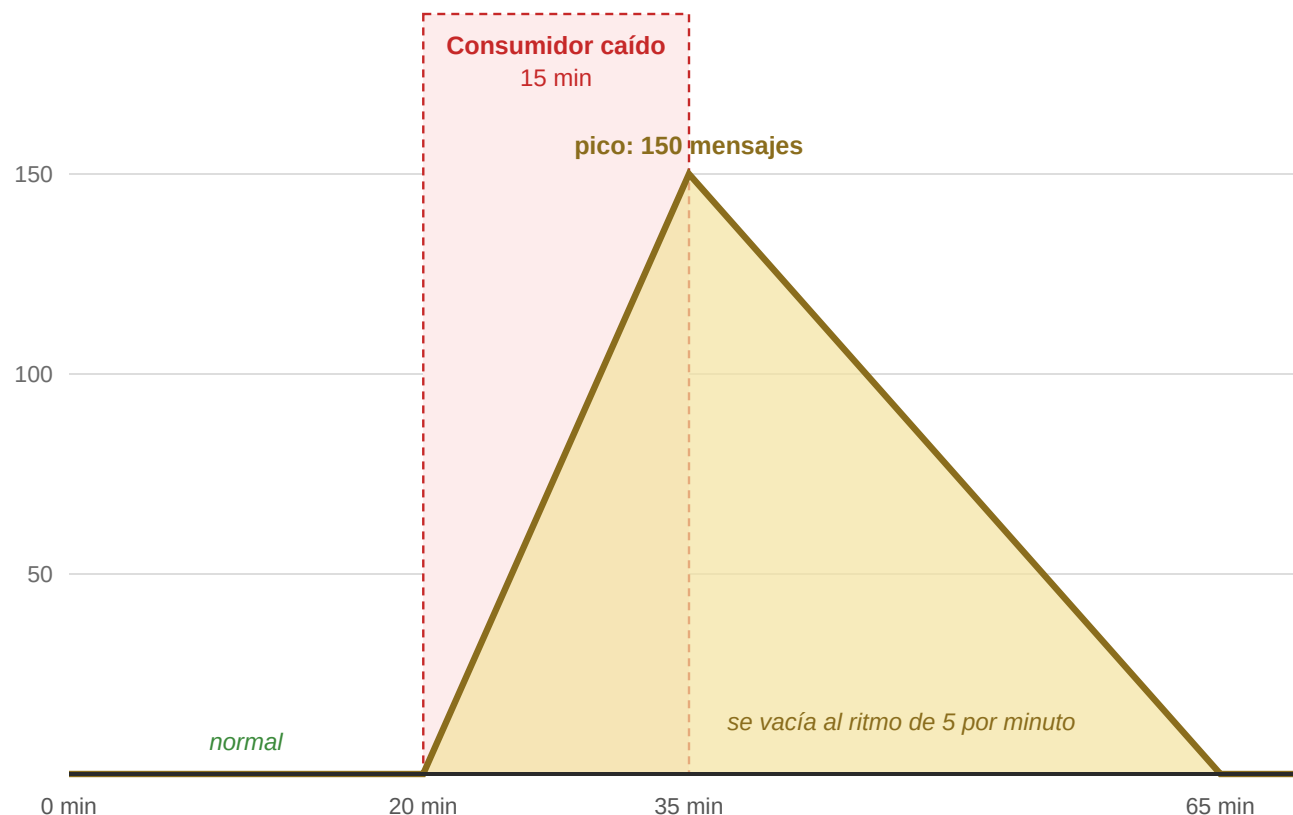
Qué aporta el intermediario



Beneficio uno: la cola absorbe la caída del consumidor

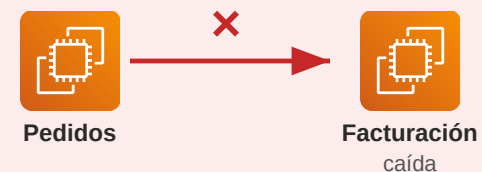
Mientras el consumidor está caído, los mensajes se acumulan; al volver, se procesan sin pérdida.

Mensajes en la cola (profundidad)



Llegan 10 mensajes por minuto; el consumidor procesa 15 por minuto (valores ilustrativos).

Sin cola (llamada directa)



Cada pedido de esos 15 minutos falla o se pierde: error 500 al usuario.

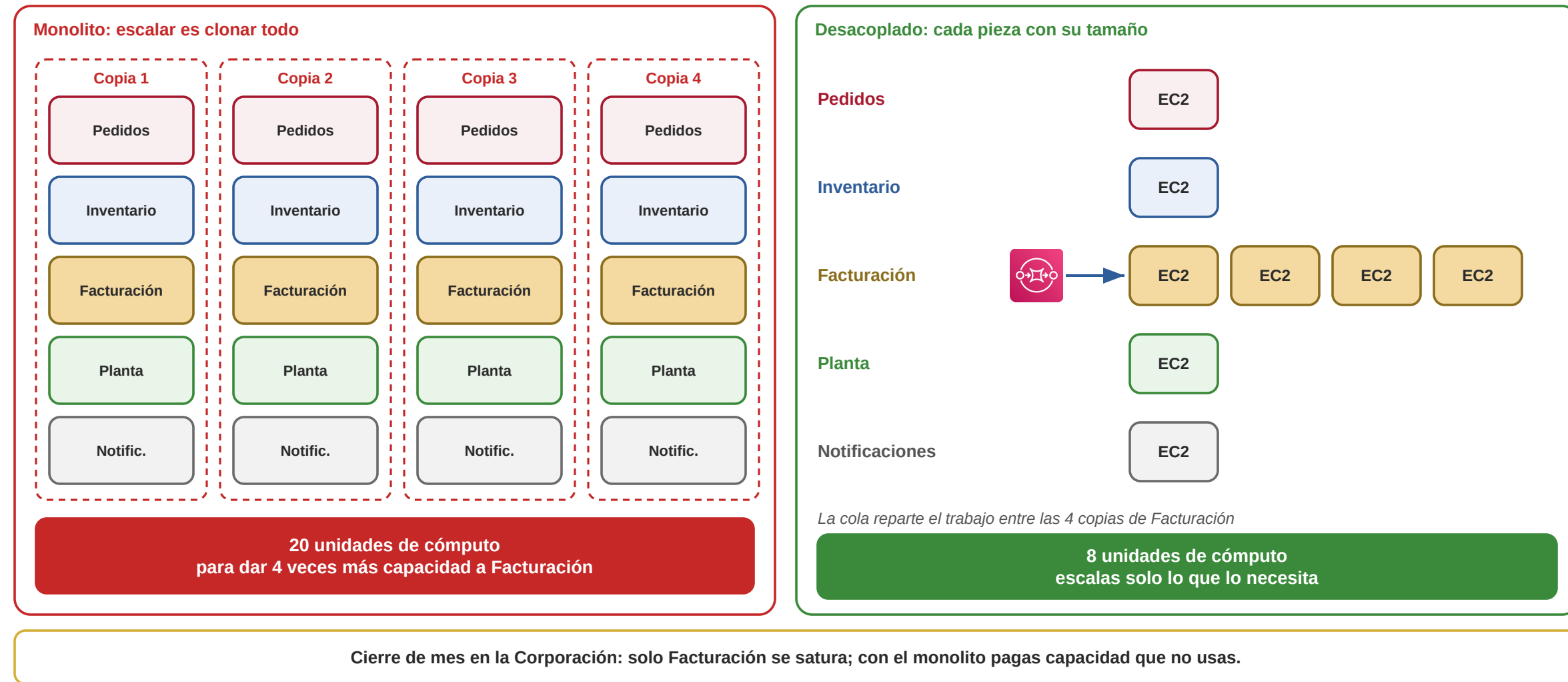
Con cola SQS



Ningún pedido se pierde: la cola retiene los mensajes (hasta 14 días) y se drena al volver.

Beneficio dos: escalar cada pieza por separado

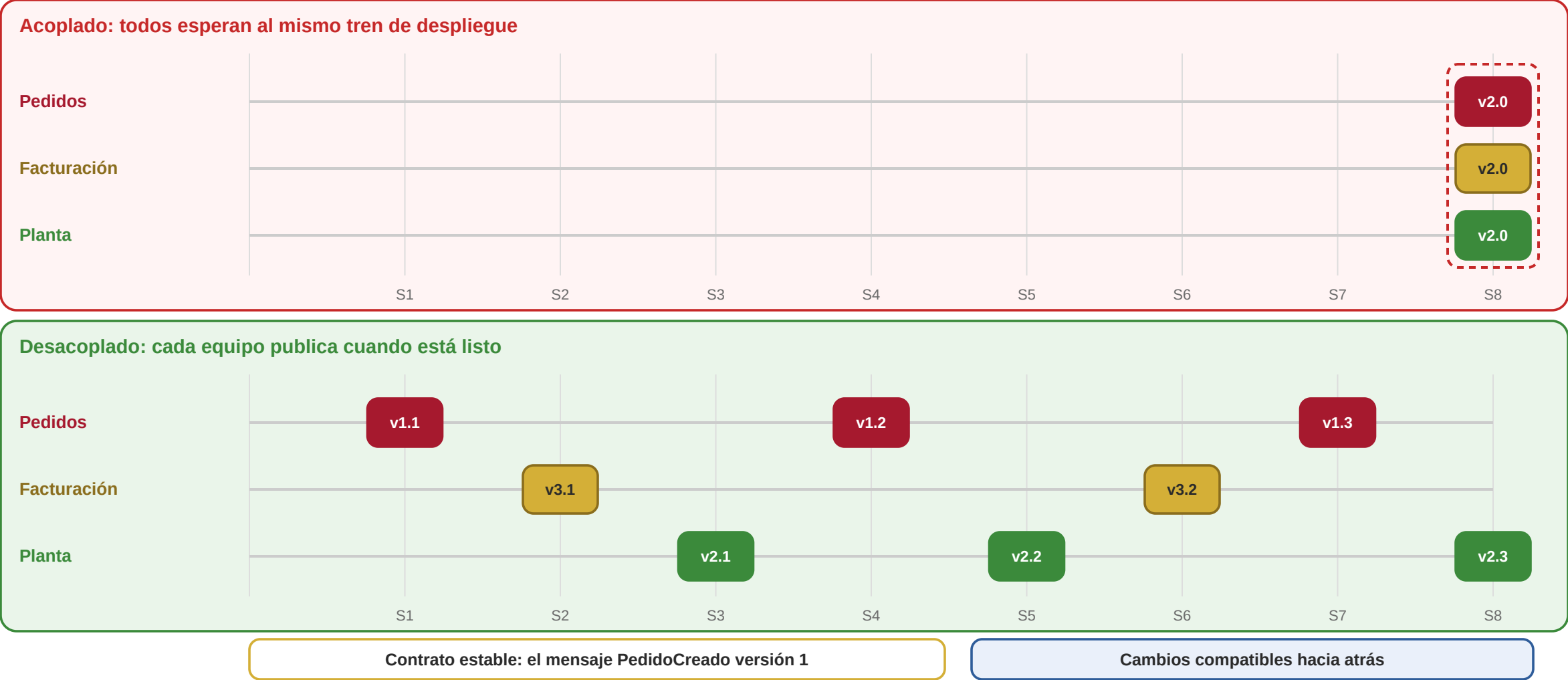
Cada servicio crece según su propia demanda en lugar de clonar la aplicación entera.



BENEFICIOS

Beneficio tres: equipos que despliegan sin pedir permiso

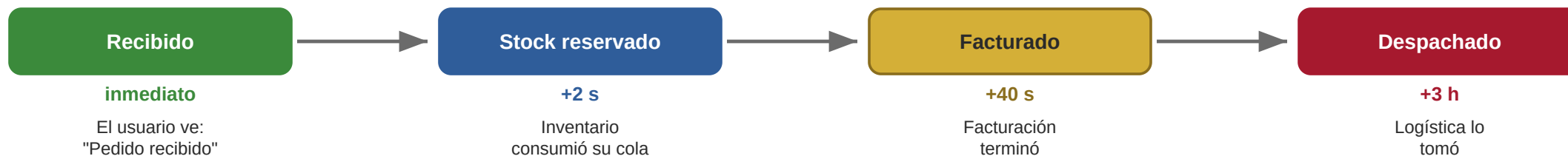
El mensaje es el contrato: mientras no cambie, cada equipo evoluciona su servicio a su ritmo.



Lo que se paga: consistencia eventual y más piezas

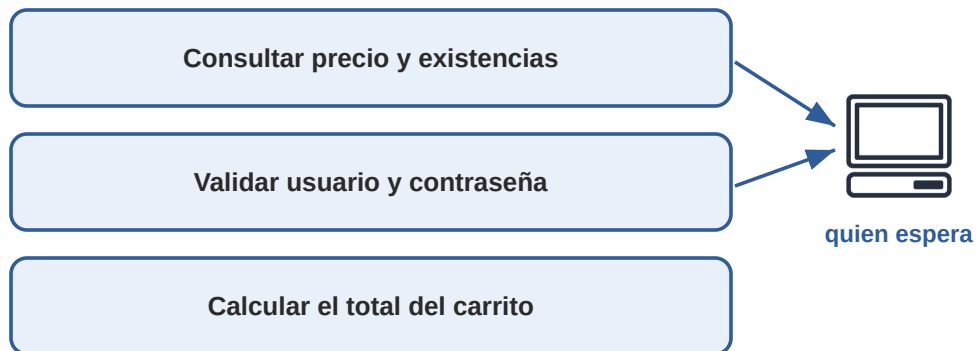
No todo debe ser asíncrono: se elige por la urgencia de la respuesta, y se acepta la complejidad.

Consistencia eventual: el pedido avanza por estados

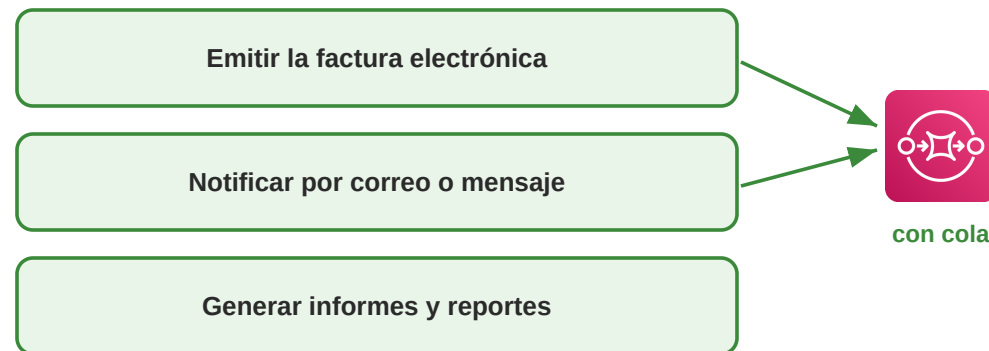


El sistema converge al estado correcto, pero no todo ocurre en el mismo instante.

Síncrono: necesito la respuesta ahora



Asíncrono: puede tardar segundos o minutos



Costos nuevos a vigilar: mensajes duplicados, orden no garantizado, y más piezas que monitorizar (idempotencia y trazabilidad: Parte 4).

Lo que queda de la Parte 1

El acoplamiento fuerte convierte un fallo local en un fallo global; el asíncrono lo contiene.

1 Acoplamiento



Cuanto menos sabe A de B,
más libres son ambos.

Se mide en dependencias

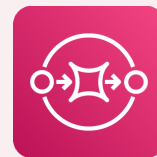
2 Cadenas síncronas



Latencia sumada, disponibilidad
multiplicada y cascadas.

Un eslabón lento tumba todo

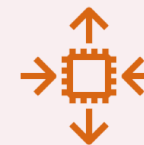
3 Asíncrono



Productor, mensaje, cola
y consumidor, sin esperar.

El mensaje espera su turno

4 Beneficios



Resiliencia, escala propia
y despliegues independientes.

A cambio de complejidad

Siguiente: Parte 2

Colas de mensajes con Amazon SQS

Estándar y FIFO · tiempo de visibilidad · sondeo largo · colas de mensajes fallidos · nivelación de carga



Colas de mensajes con Amazon SQS

Parte 2 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

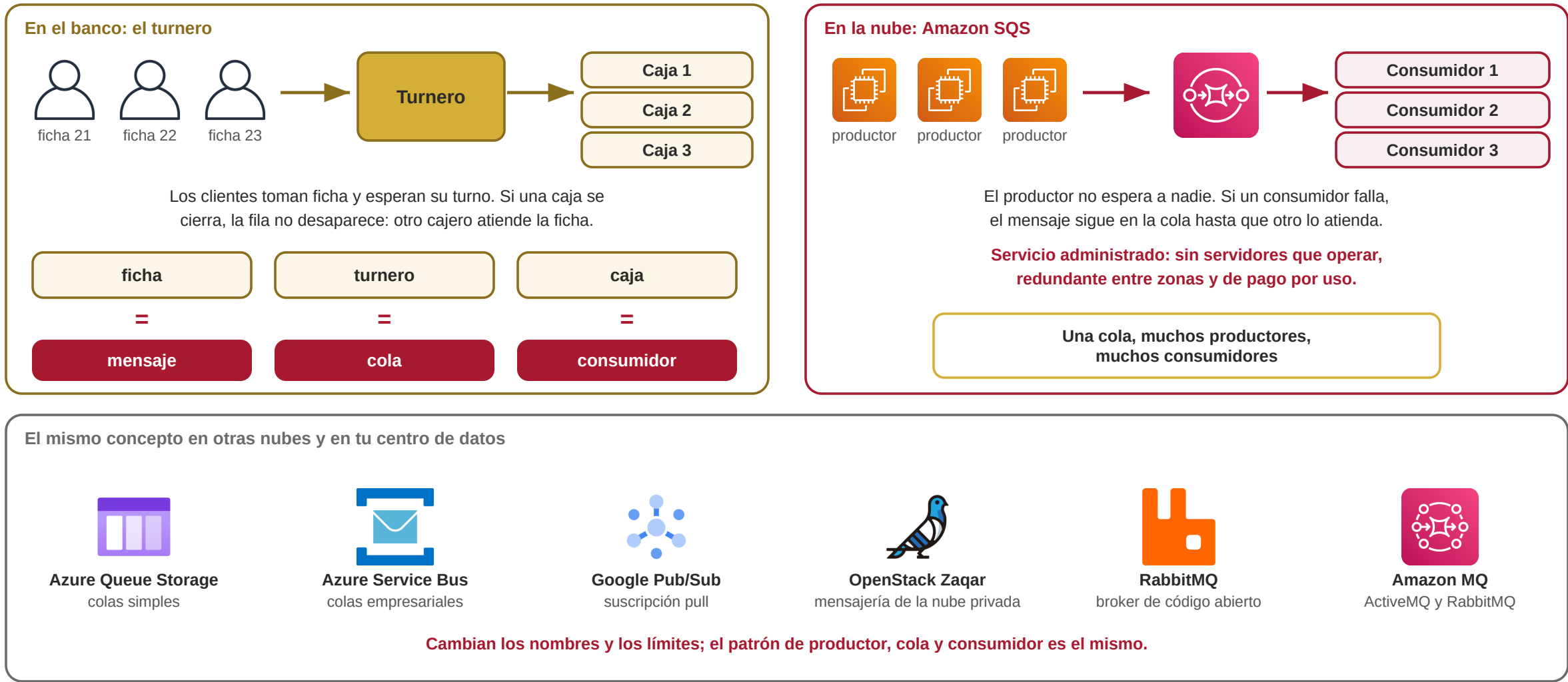
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Cómo una cola administrada entrega, protege y reparte el trabajo entre consumidores.

CONCEPTO

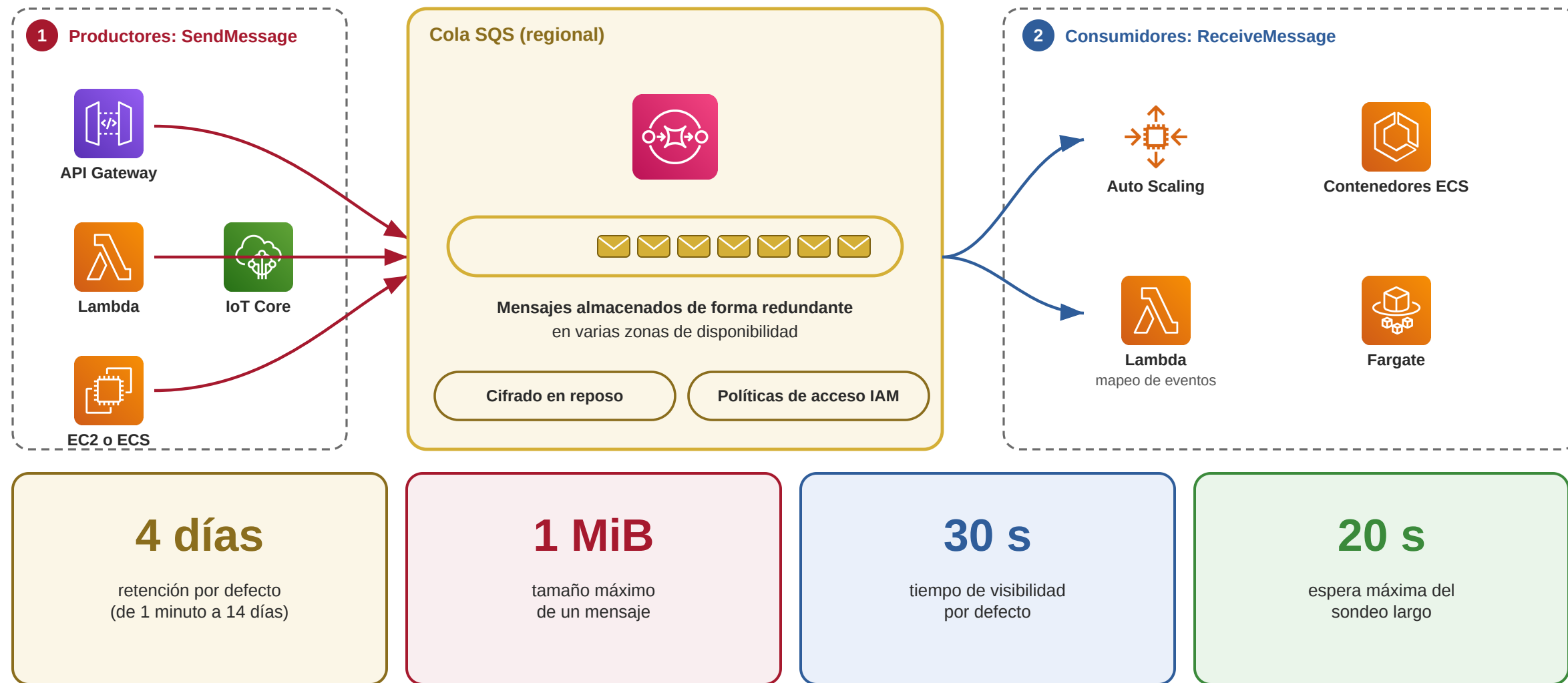
Una cola de turnos y una cola de mensajes

La cola desacopla a quien produce trabajo de quien lo atiende, igual que el turnero de un banco.



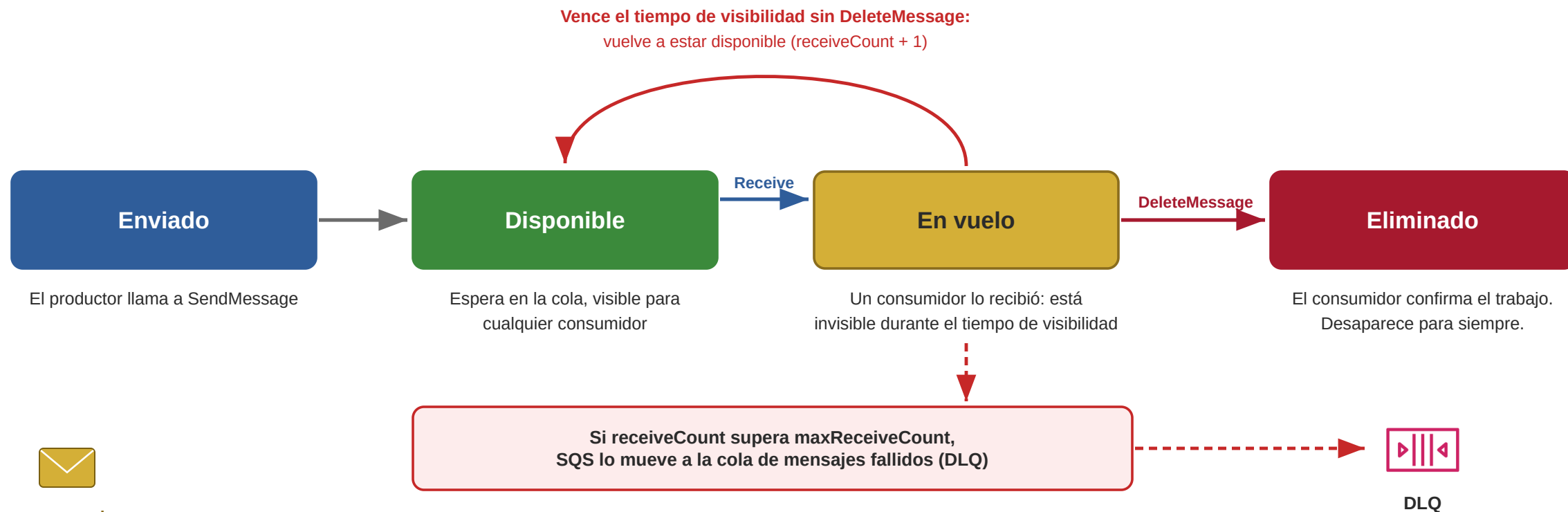
Anatomía de Amazon SQS

Productores envían, la cola guarda de forma redundante y los consumidores toman el trabajo.



Ciclo de vida de un mensaje en SQS

Un mensaje no desaparece al leerse: queda invisible hasta que el consumidor lo confirma borrándolo.




un mensaje
recorre
estos estados

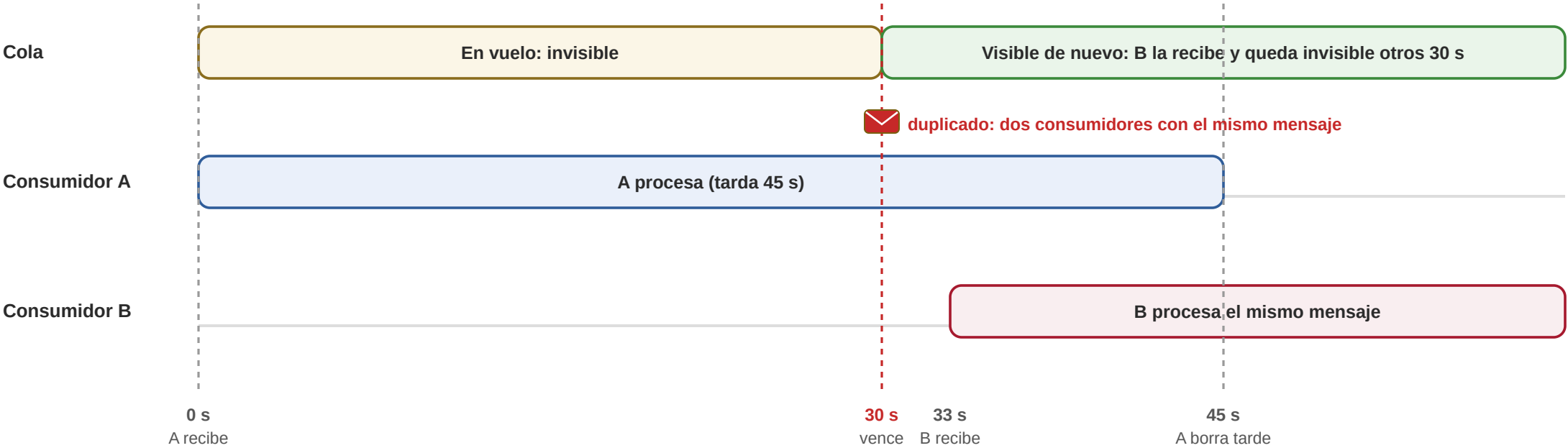
Garantía clave: SQS entrega al menos una vez. Un mensaje puede reaparecer;
por eso el consumidor debe poder procesarlo dos veces sin daño (idempotencia, Parte 4).

Eliminado, expirado por retención o movido a la DLQ: los tres finales posibles.

Tiempo de visibilidad: el candado temporal

Si el consumidor tarda más que el tiempo de visibilidad, otro recibe el mismo mensaje.

Línea de tiempo con tiempo de visibilidad de 30 s, y un procesamiento que tarda 45 s



Cómo evitarlo

- Visibilidad mayor que el tiempo de procesamiento
- ChangeMessageVisibility: extender si se necesita más
- Rango: de 0 s a 12 horas (30 s por defecto)
- Consumidor idempotente: un duplicado no hace daño

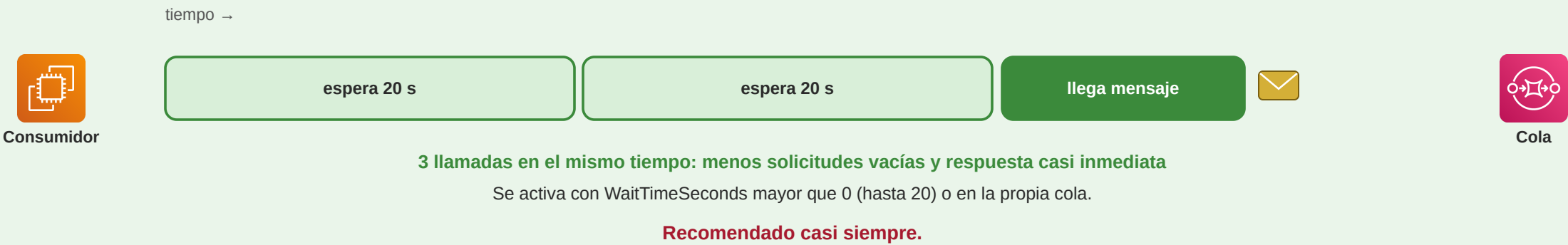
Sondeo corto y sondeo largo

El sondeo largo espera hasta 20 segundos: menos llamadas vacías, menor costo y menos latencia.

Sondeo corto (short polling): responde de inmediato, aunque esté vacío



Sondeo largo (long polling): espera hasta 20 s a que llegue un mensaje



SQS estándar: máximo rendimiento, orden aproximado

La cola estándar escala sin límite práctico, a cambio de duplicados ocasionales y orden no garantizado.

Productor envía en este orden



Productor

Cola estándar: la entrega se reparte entre servidores



Consumidores

Recibido



← orden distinto y el mensaje 2 llega dos veces

Rendimiento casi ilimitado

Miles de mensajes por segundo
sin configurar capacidad

Ideal para volumen alto

Entrega al menos una vez

Puede haber duplicados:
el consumidor debe tolerarlos

Hazlo idempotente

Orden de mejor esfuerzo

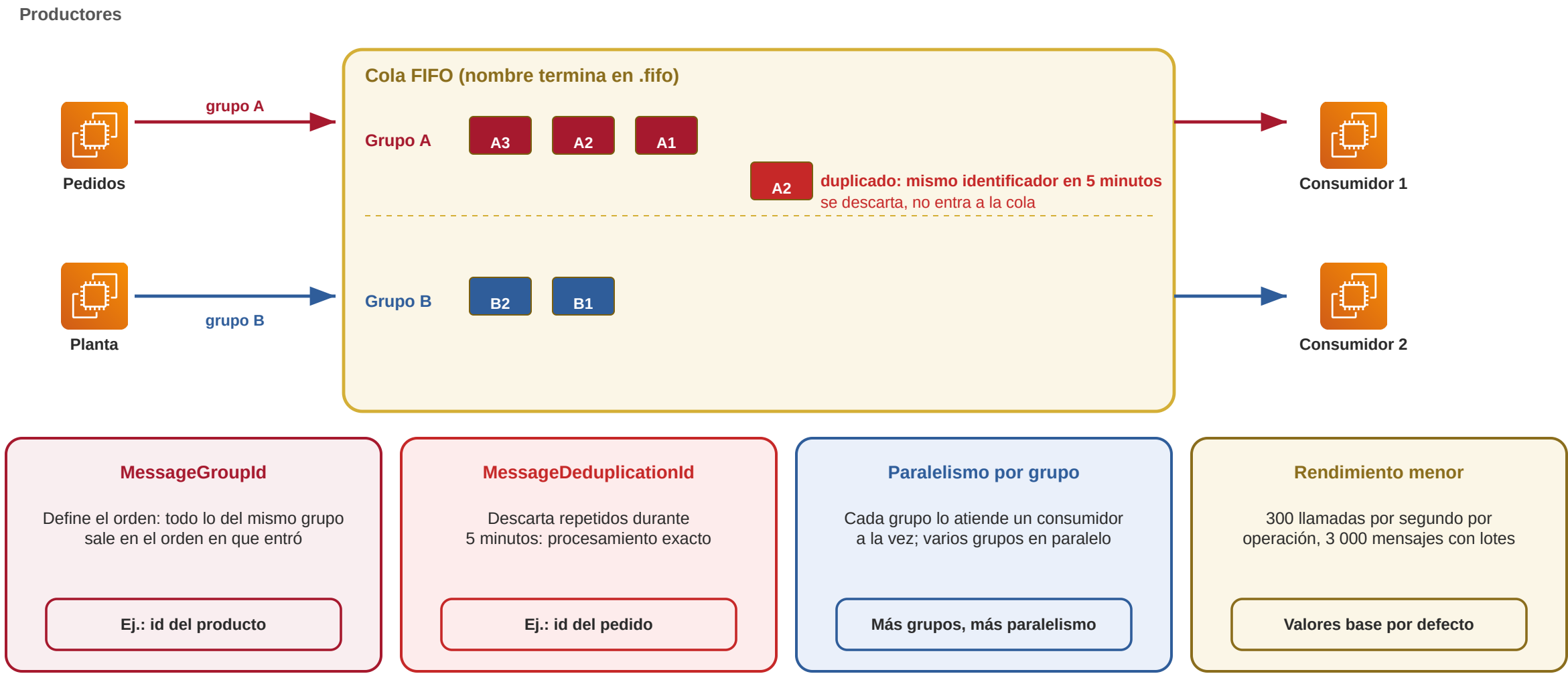
Suele respetar el orden,
pero no lo garantiza

No dependas del orden

Por defecto elige estándar: notificaciones, procesamiento de imágenes, registros, tareas independientes entre sí.

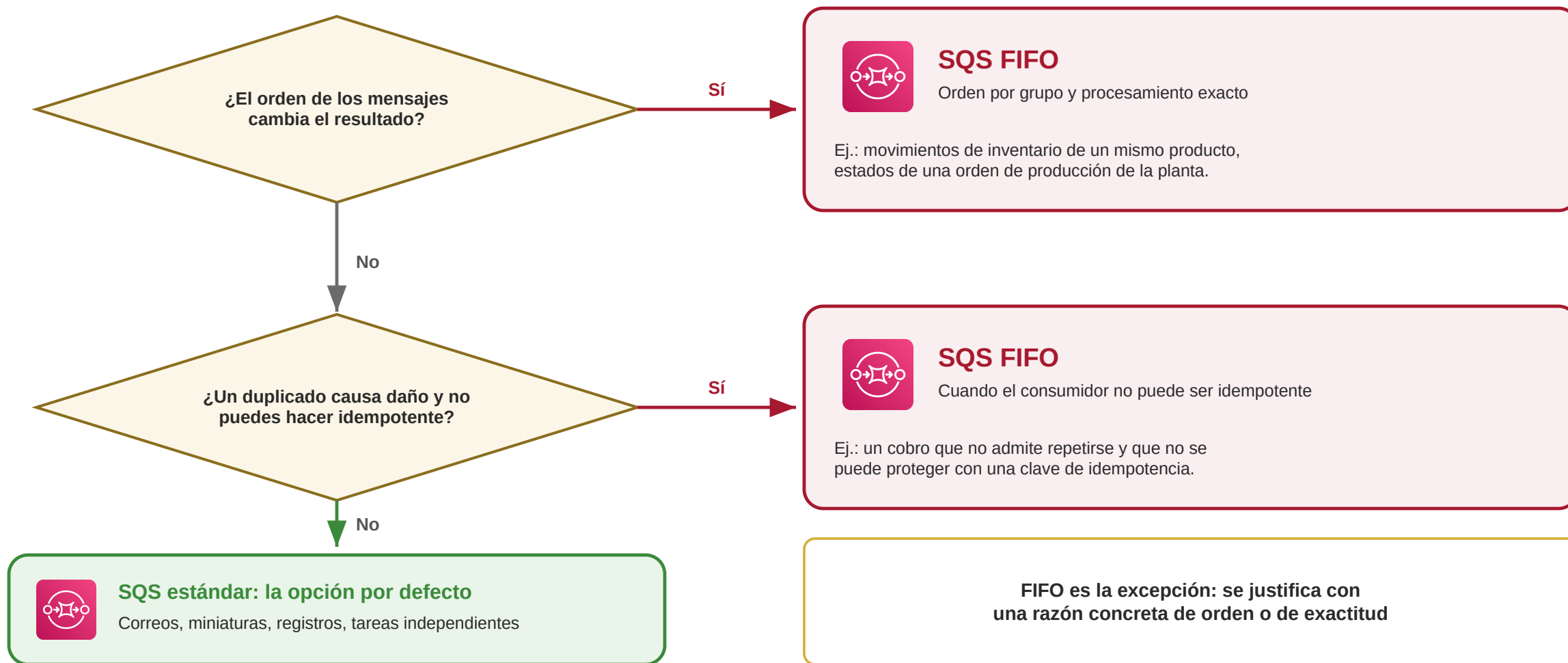
SQS FIFO: orden estricto y sin duplicados

FIFO garantiza el orden dentro de cada grupo de mensajes y descarta duplicados, con menor rendimiento.



Estándar o FIFO: cómo elegir

Empieza por estándar; pasa a FIFO solo cuando el orden o la exactitud lo exijan de verdad.



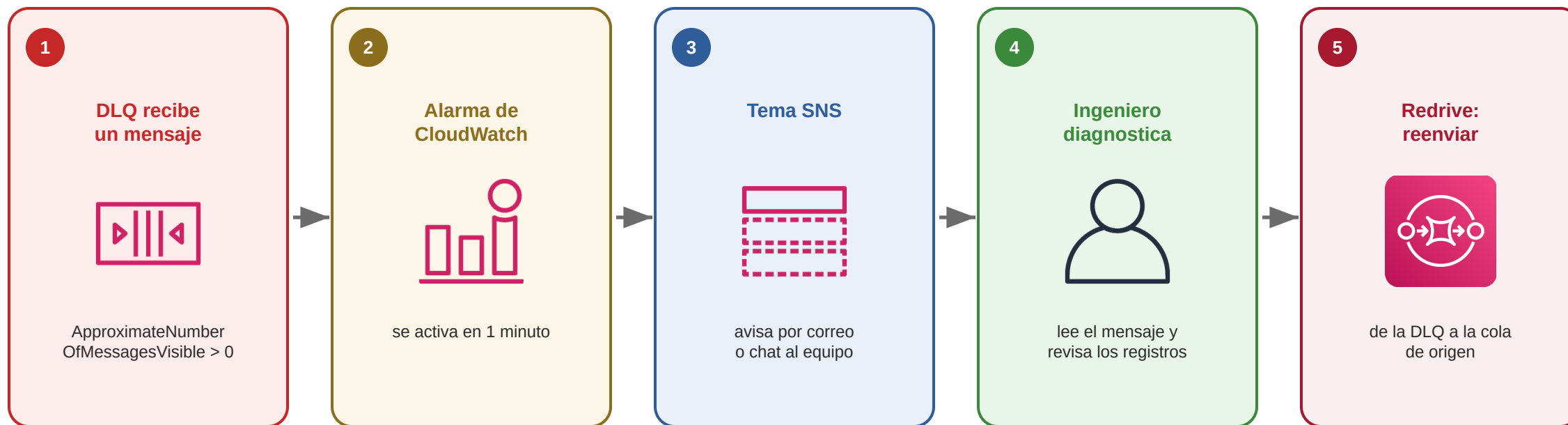
Colas de mensajes fallidos (DLQ): aparcar al mensaje veneno

Tras superar maxReceiveCount, SQS aparta el mensaje a otra cola para que no bloquee a los demás.



Operar la DLQ: alarma, diagnóstico y reproceso

Una cola de mensajes fallidos solo sirve si alguien la vigila y sabe reprocesar lo aparcado.



Reproceso seguro: primero corrige el código o los datos; después mueve los mensajes de vuelta (consola o `StartMessageMoveTask`)

Una DLQ con mensajes es una señal de alerta, no un cementerio:

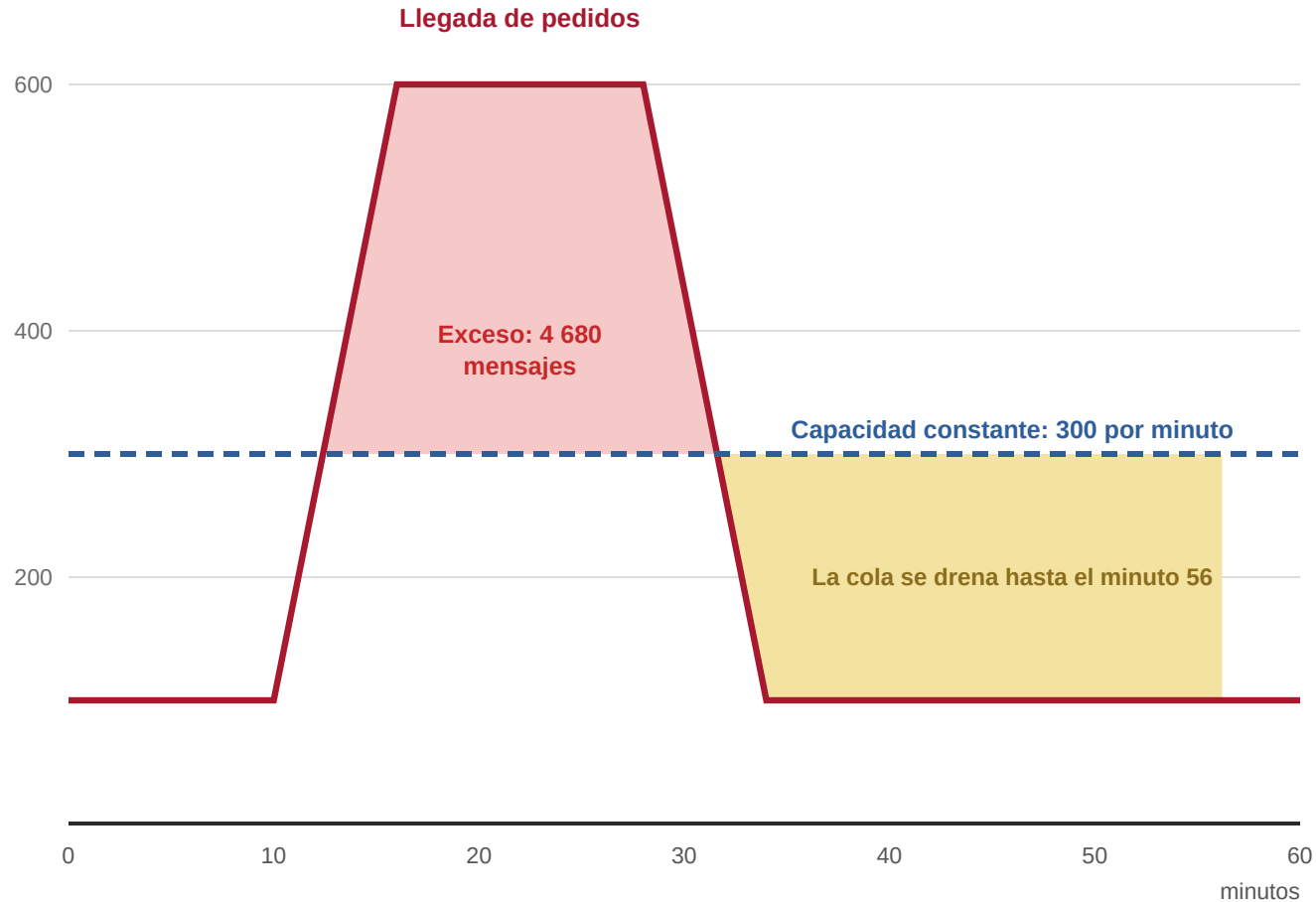
si nadie la vigila, los pedidos pendientes se pierden en silencio y el negocio no se entera.

Mide también `ApproximateAgeOfOldestMessage` en la cola de origen: indica cuánto lleva esperando el mensaje más antiguo.

Nivelación de carga: absorber el pico

La cola convierte un pico de llegada en un flujo constante que el consumidor sí puede atender.

Mensajes por minuto



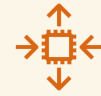
Nivelación de carga



Pico



Cola



Capacidad fija

El pico no pasa a la base de datos ni al ERP: la cola lo convierte en un flujo estable.

Dimensionas para el promedio,
no para el pico

Sin cola: el exceso se rechaza
o tumba al consumidor

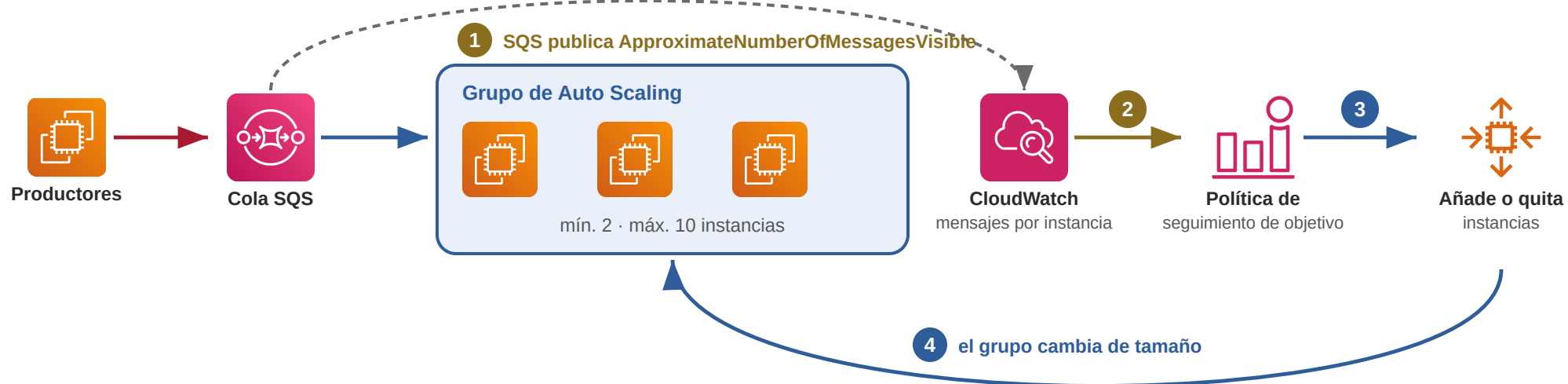
Con cola: el exceso espera y se
procesa cuando hay capacidad

Caso: cierre de mes de la Corporación.

Cifras ilustrativas.

Escalar consumidores según la profundidad de la cola

La longitud de la cola mide la demanda real; con ella el grupo de instancias crece y se encoge solo.



Cómo fijar el objetivo (ejemplo)

Cada instancia procesa 5 mensajes por segundo
Latencia aceptable: 60 segundos

Objetivo por instancia = $5 \times 60 = 300$ mensajes

$2\,400 \text{ mensajes} \div 300 = 8 \text{ instancias}$

La métrica de escala es mensajes en cola por instancia, no la CPU.

Cifras ilustrativas.



Lo que queda de la Parte 2

Con SQS el trabajo espera seguro, se reparte entre consumidores y se vigila con alarmas.

1 Cola SQS



Servicio administrado:
enviar, recibir, borrar.

Guarda y entrega

2 Visibilidad y sondeo



Candado temporal y
sondeo largo de 20 s.

Menos duplicados y costo

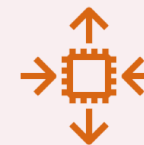
3 Estándar o FIFO



Rendimiento y escala frente
a orden y exactitud.

Elige con una pregunta

4 DLQ y nivelación



Aparcar al veneno, absorber
picos, escalar por cola.

Resiliencia en acción

Siguiente: Parte 3

Publicación y suscripción, y eventos

SNS · fan-out hacia colas SQS · EventBridge y sus reglas · Azure Service Bus y Event Grid



Publicación y suscripción, y eventos

Parte 3 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

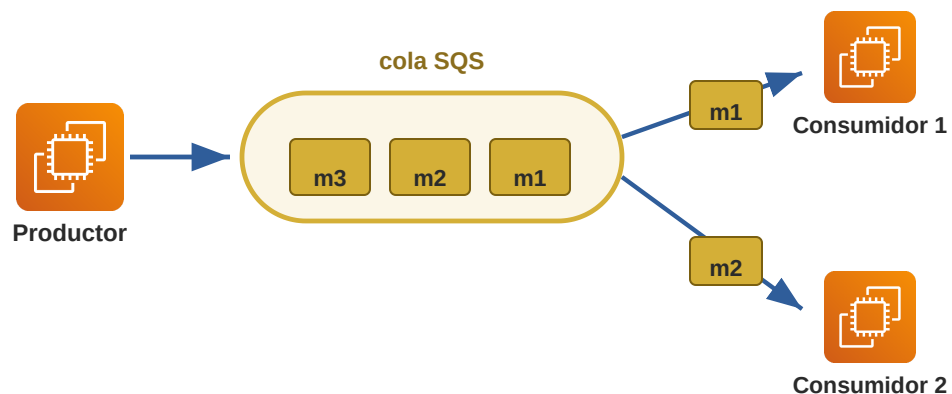
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Cómo un solo suceso llega a muchos sistemas sin que el productor los conozca: SNS y EventBridge.

De uno a uno, a uno a muchos

La cola reparte tareas entre consumidores; publicación y suscripción difunde una noticia a todos.

Cola: cada mensaje va a UN consumidor



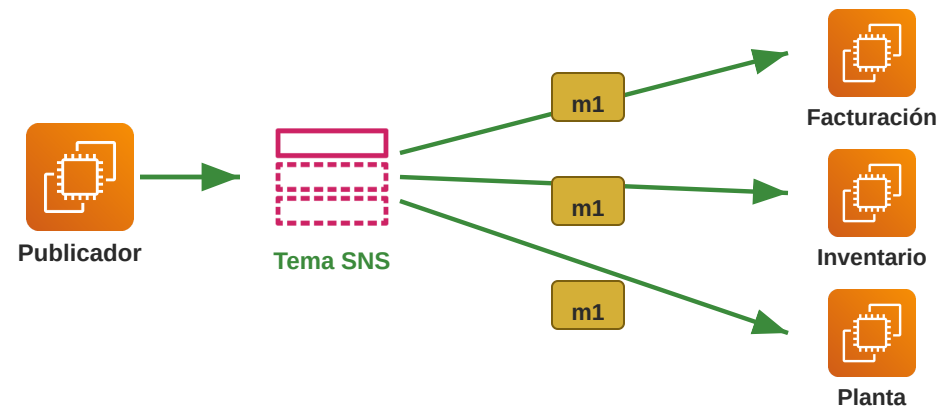
m1 la recibe solo el consumidor 1,
m2 solo el consumidor 2: se reparten el trabajo.

Consumidores que compiten: reparto de trabajo

Úsala cuando el mensaje es una tarea
que debe ejecutarse una sola vez.

Ejemplo: emitir una factura.

Publicación y suscripción: a TODOS los suscriptores



m1 la reciben los tres: cada uno
tiene su propia copia del mensaje.

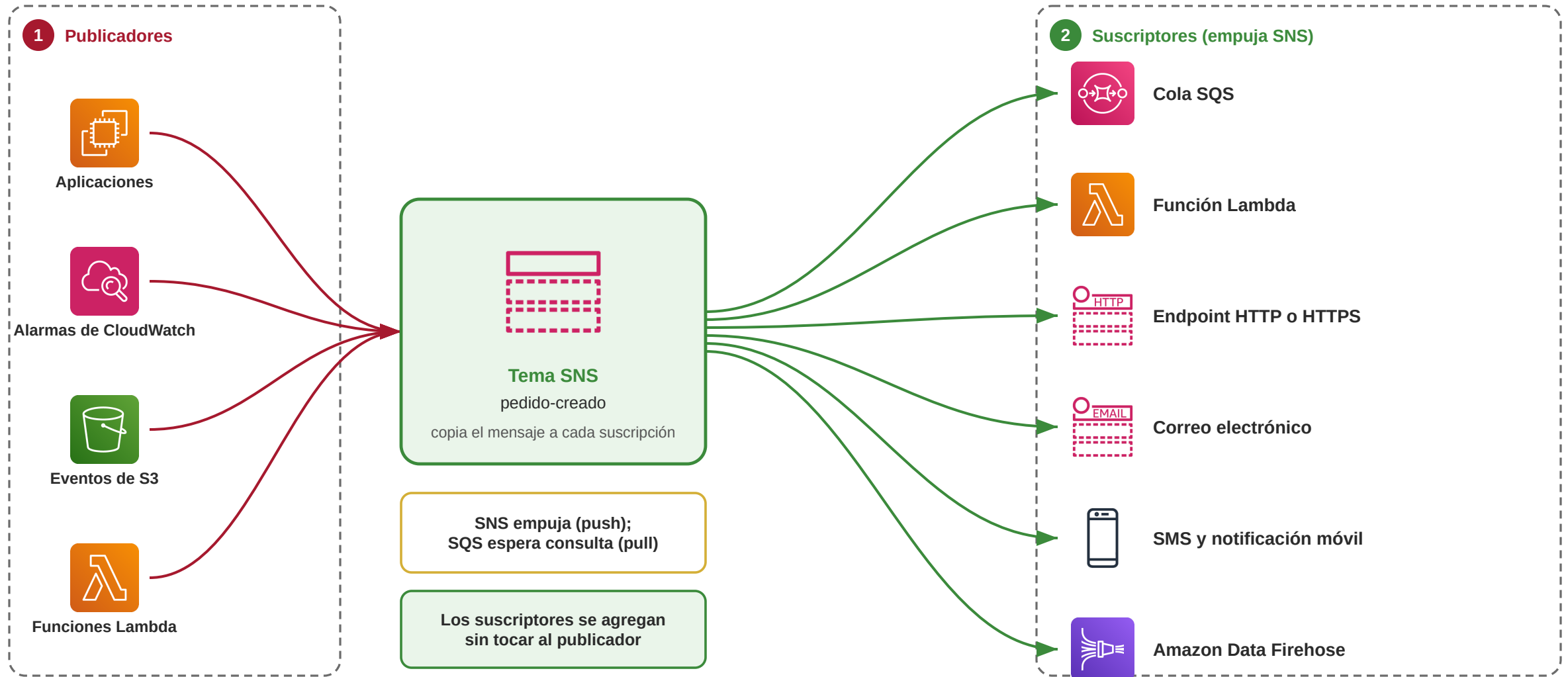
Difusión: un mensaje, muchas copias

Úsala cuando el mensaje es una noticia
que interesa a varios sistemas.

Ejemplo: se creó un pedido.

Amazon SNS: temas y suscriptores

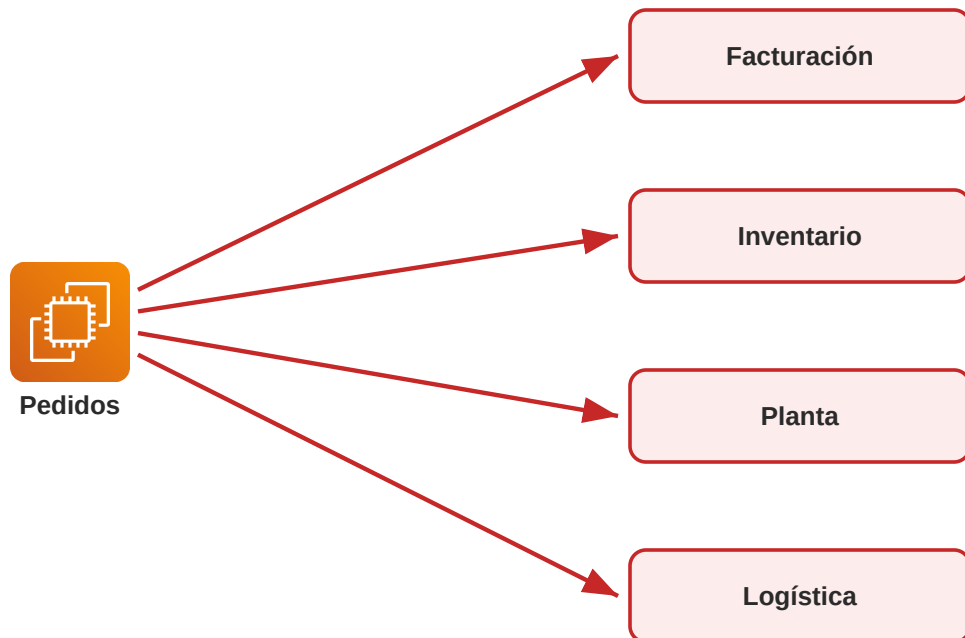
El publicador envía una vez al tema; SNS entrega una copia a cada suscripción, por empuje.



El problema que resuelve el fan-out

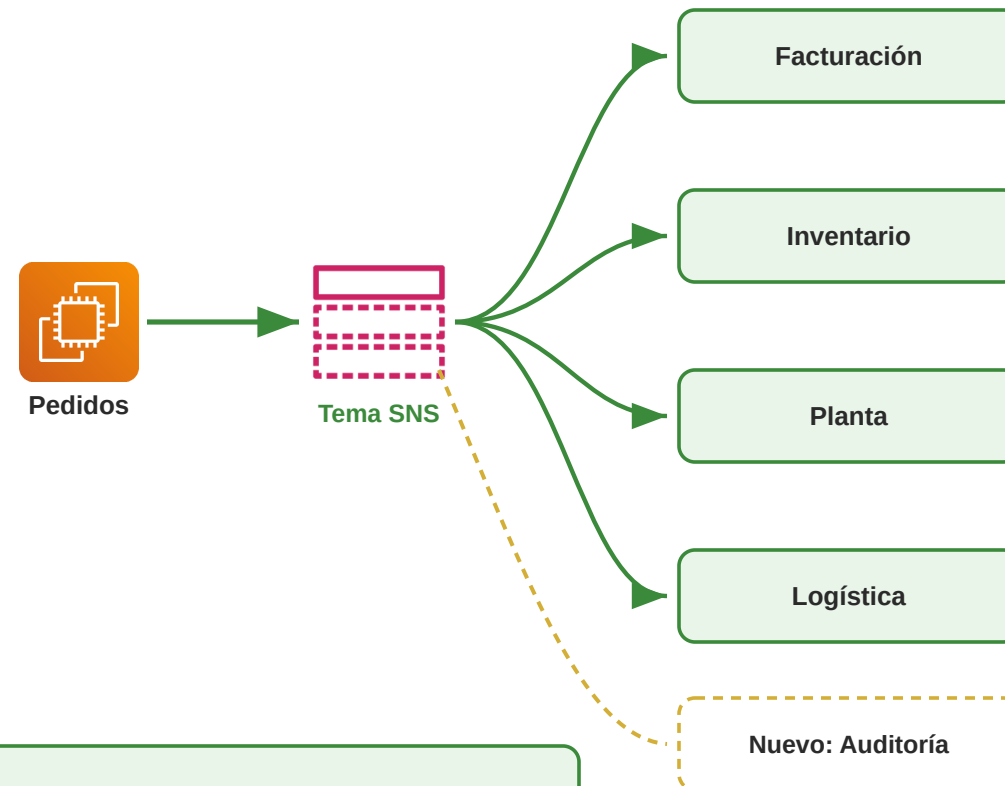
Un solo suceso, varios interesados: publicar una vez evita que Pedidos conozca a todos.

Antes: Pedidos llama a cada servicio



Cada nuevo interesado obliga a modificar, probar y desplegar el servicio de Pedidos

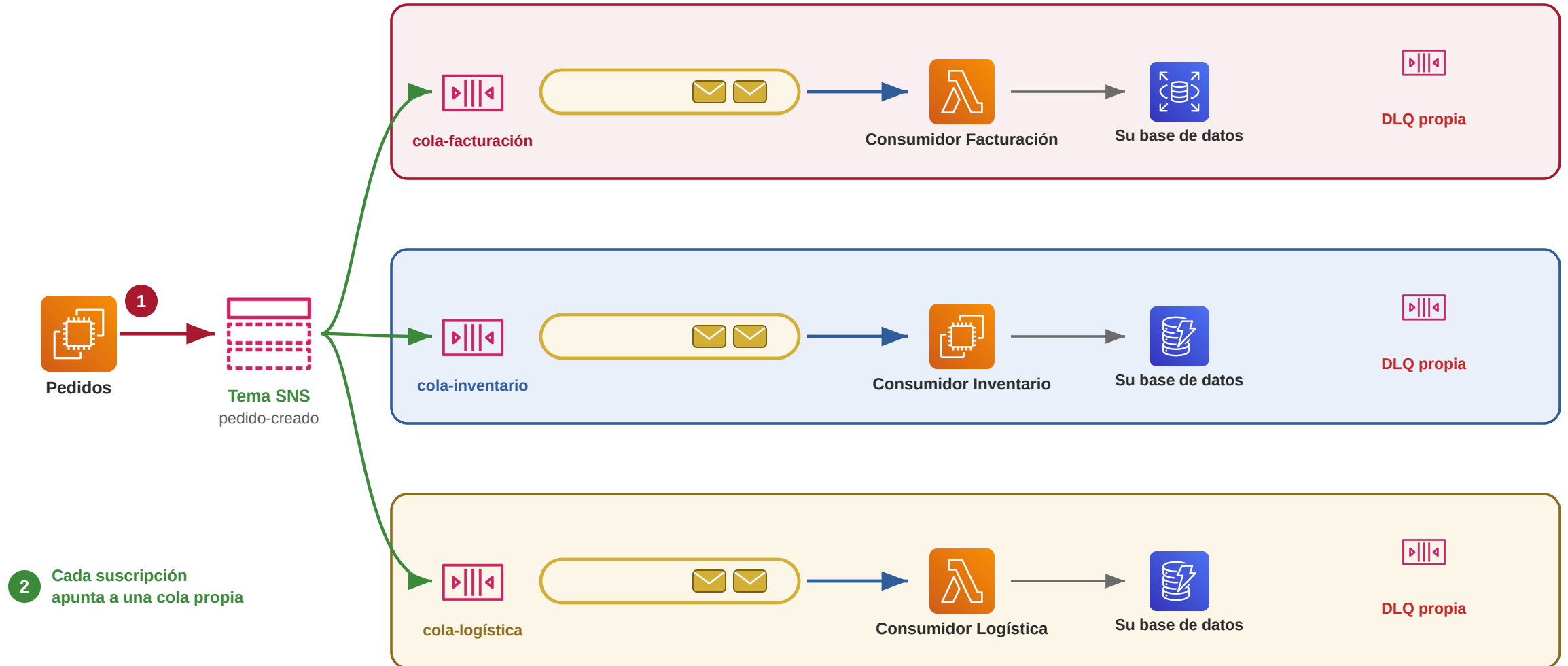
Después: Pedidos publica una sola vez



Se agrega un suscriptor sin tocar a Pedidos

Patrón fan-out: SNS hacia varias colas SQS

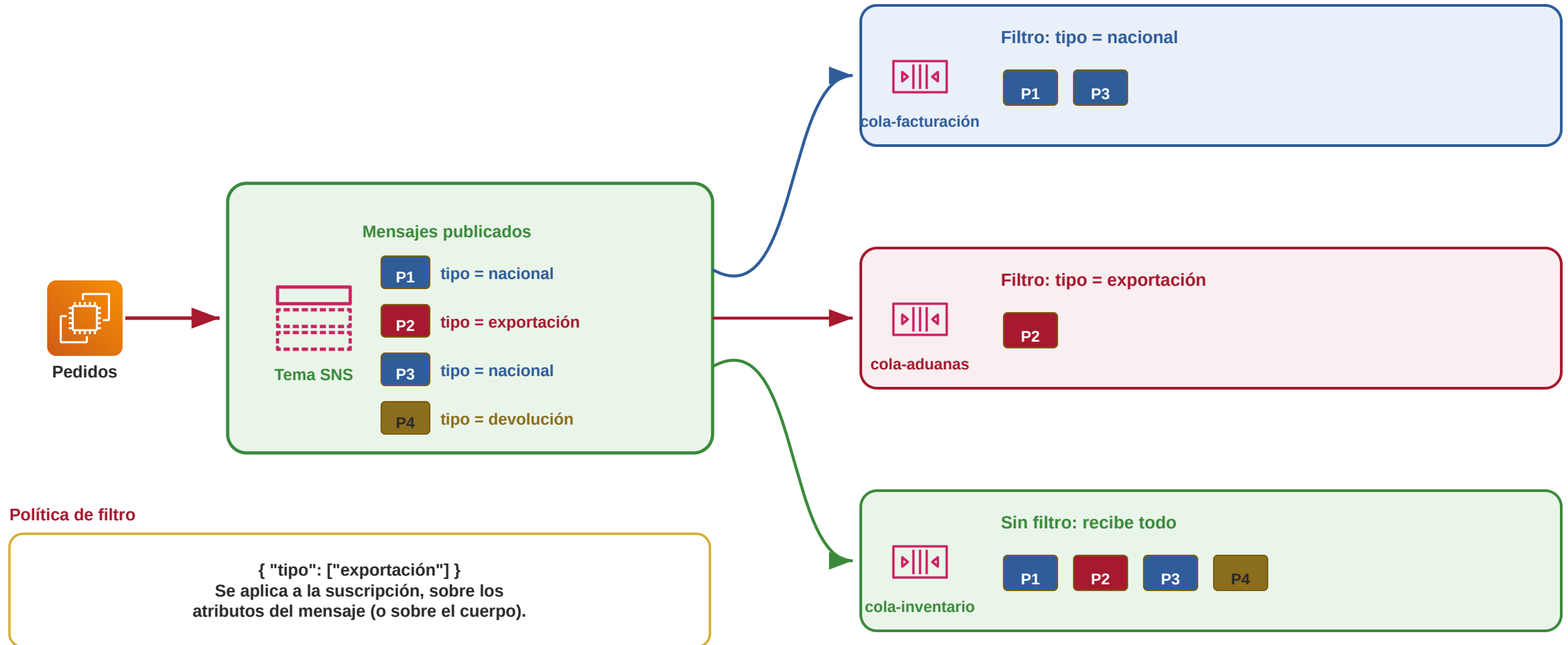
El tema difunde, y cada cola absorbe el trabajo de su consumidor a su propio ritmo.



La política de la cola permite que el tema le envíe mensajes. Si un consumidor se cae, su cola retiene el trabajo sin afectar a los demás.

Filtros de suscripción: cada cola recibe lo suyo

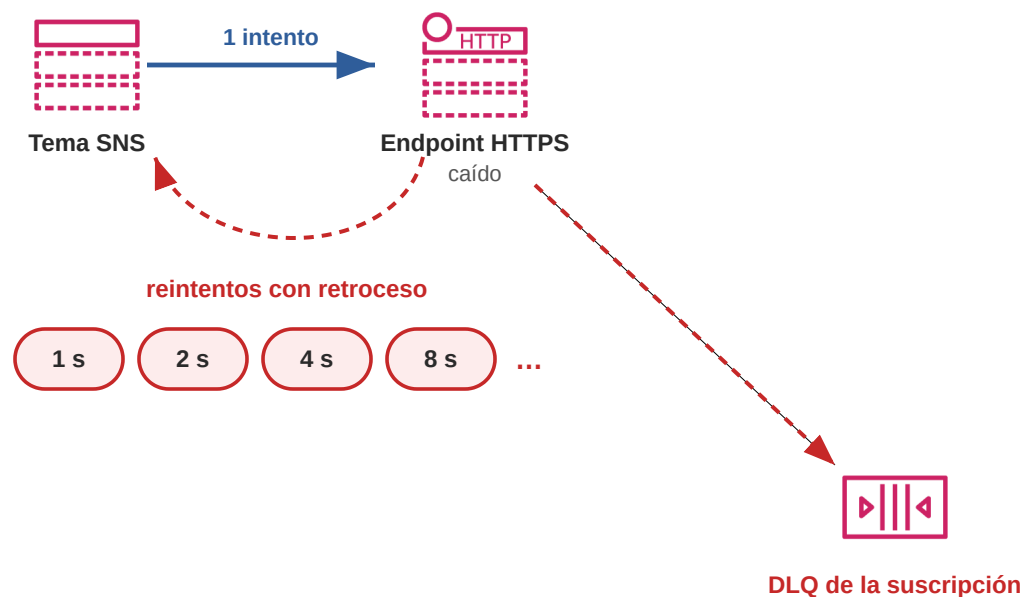
Una política de filtro en la suscripción descarta lo que al suscriptor no le interesa.



SNS FIFO y entrega confiable

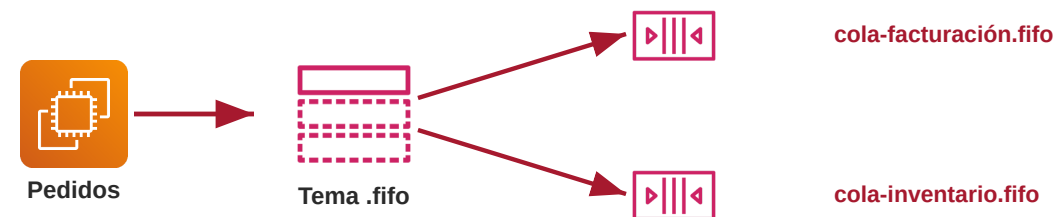
SNS reintenta con retroceso y desvía lo no entregado a una DLQ; el tema FIFO conserva el orden.

Entrega confiable: reintentos y DLQ



Si agota los reintentos de la política de entrega,
SNS guarda el mensaje en una cola SQS (DLQ)
y no se pierde.

SNS FIFO: orden hacia las colas FIFO



A1 A2 A3

El orden del grupo A se mantiene de punta a punta,
y los duplicados se descartan.

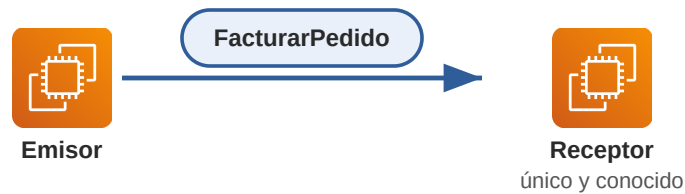
Mismo MessageGroupId y deduplicación que en
SQS FIFO, ahora desde el tema

Los temas FIFO entregan a colas SQS FIFO:
se usan cuando el orden importa para varios
consumidores a la vez

Eventos: algo ocurrió, y a quién le importa

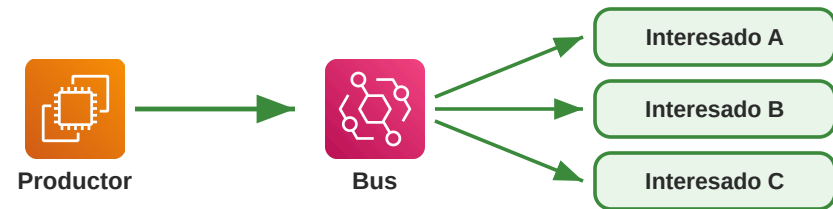
Un evento describe un hecho pasado; quien lo emite no espera respuesta ni conoce a los oyentes.

Comando: una orden dirigida



Verbo en imperativo: espera un resultado

Evento: un hecho ya ocurrido



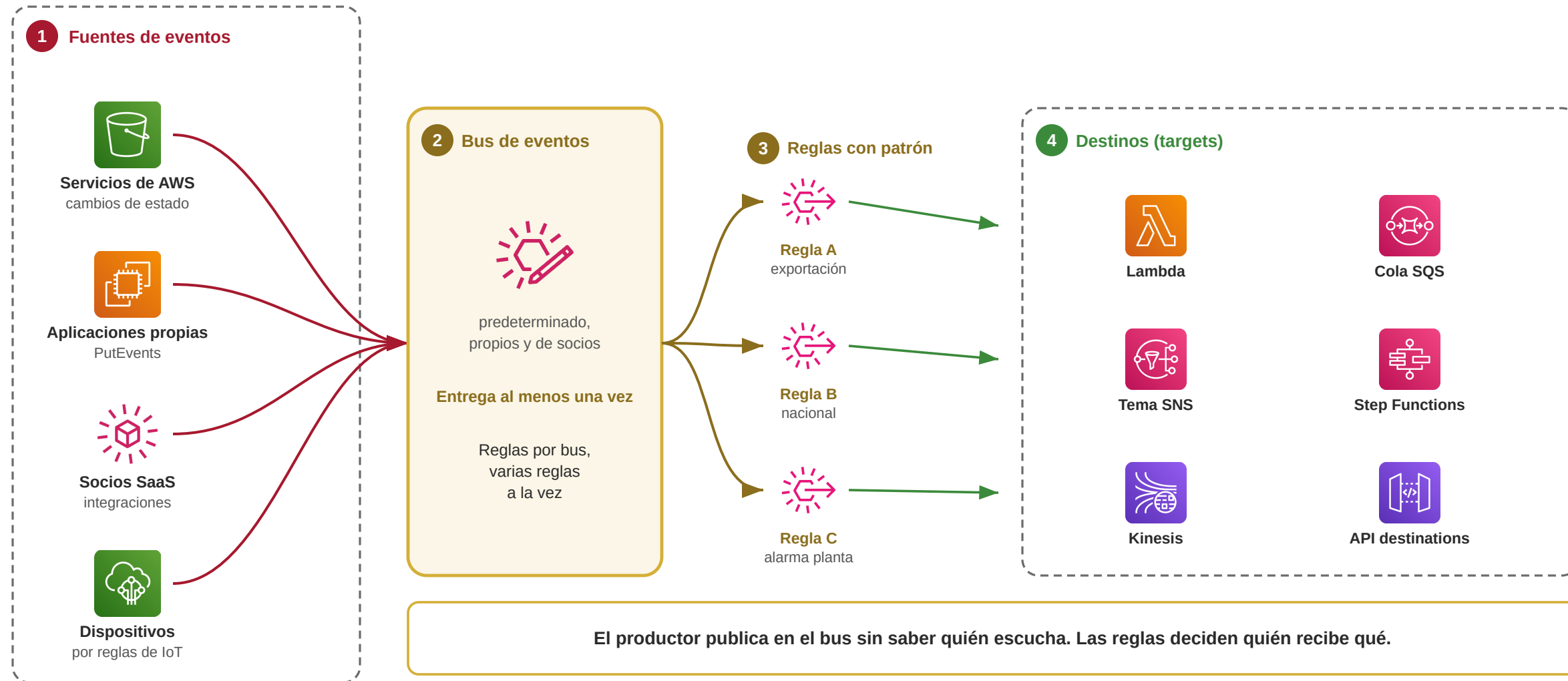
Verbo en pasado: no espera nada ni sabe quién escucha

Anatomía de un evento en EventBridge (JSON)

version "0"	esquema del sobre
id "6a7e8feb-..."	identificador único
detail-type "PedidoCreado"	qué ocurrió (pasado)
source "cic.pedidos"	quién lo emitió
account · region "123456789012" · "us-east-1"	cuenta y región
time "2026-10-12T14:03:07Z"	cuándo ocurrió
detail { "pedido": "CIC-1042", "tipo": "exportación", "total": 18400000 }	los datos del negocio

Amazon EventBridge: bus de eventos con reglas

Eventos de AWS, de tus aplicaciones y de socios entran a un bus; las reglas los enrutan a destinos.



Reglas de EventBridge: patrones que enrutan

La regla compara el evento con su patrón; solo los que coinciden en todo llegan al destino.

Patrón de la regla (JSON)

```
{
  "source": ["cic.pedidos"],
  "detail-type": ["PedidoCreado"],
  "detail": {
    "tipo": ["exportación"],
    "total": [{ "numeric": [ ">", 10000000 ] }]
  }
}
```

Operadores de contenido

igualdad exacta

prefix y suffix

numeric (rangos)

exists

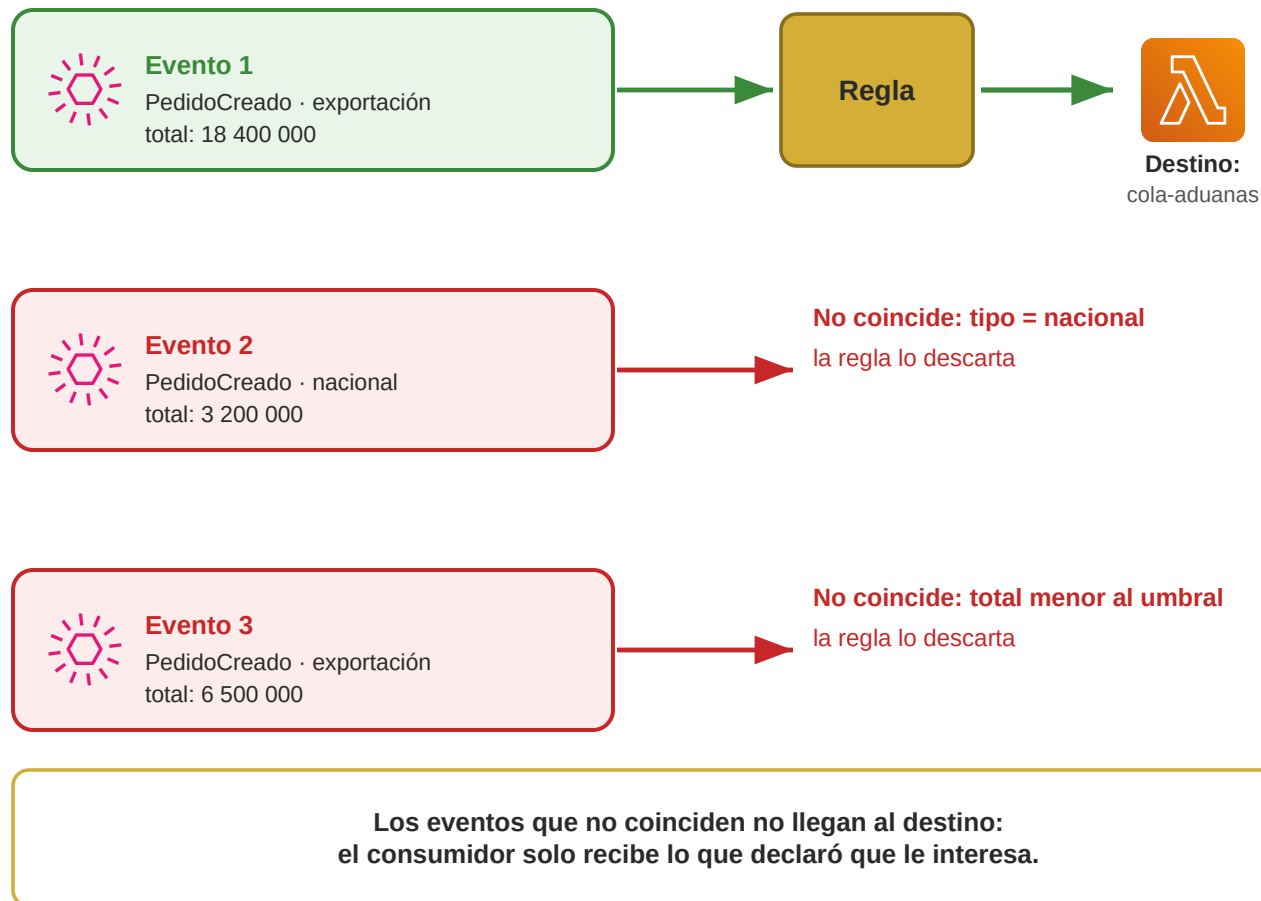
anything-but

or entre valores

Solo se activa si **TODOS** los campos del patrón coinciden.

Un campo ausente en el patrón significa "no me importa".

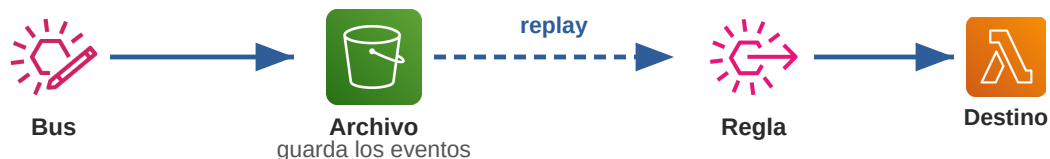
Ejemplo: exportaciones de más de 10 millones de pesos.



Archivo, esquemas, Pipes y Scheduler

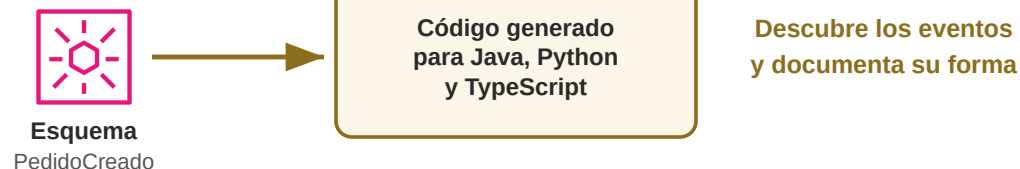
EventBridge no solo enruta: guarda, documenta, conecta y programa eventos.

Archivo y reproducción (archive and replay)



Reproduces eventos pasados tras corregir un error o probar un consumidor nuevo con datos reales

Registro de esquemas (schema registry)



El contrato del evento queda escrito y versionado

EventBridge Pipes: conectar de punto a punto



Sin código de pegamento: lee de un origen, transforma y entrega a un destino

EventBridge Scheduler: tareas programadas

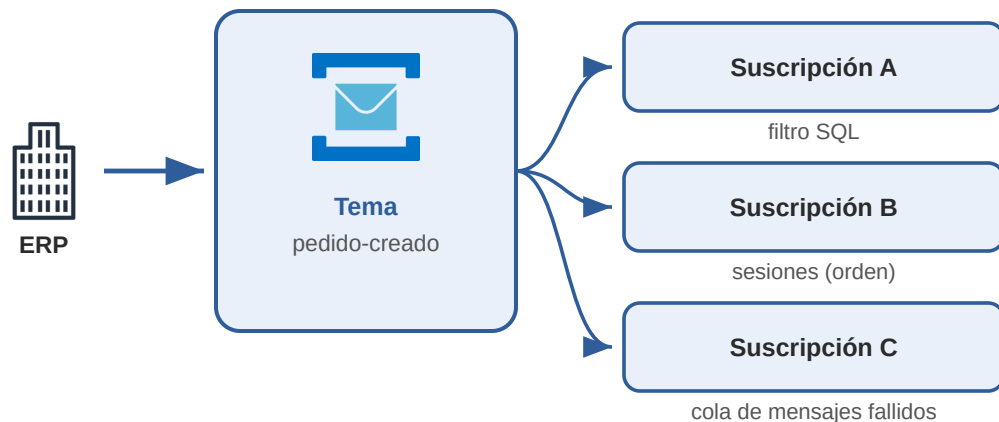


Cierre de mes cada día 30 a las 6 p. m.: un evento programado dispara el cierre contable

El mismo patrón en Azure: Service Bus y Event Grid

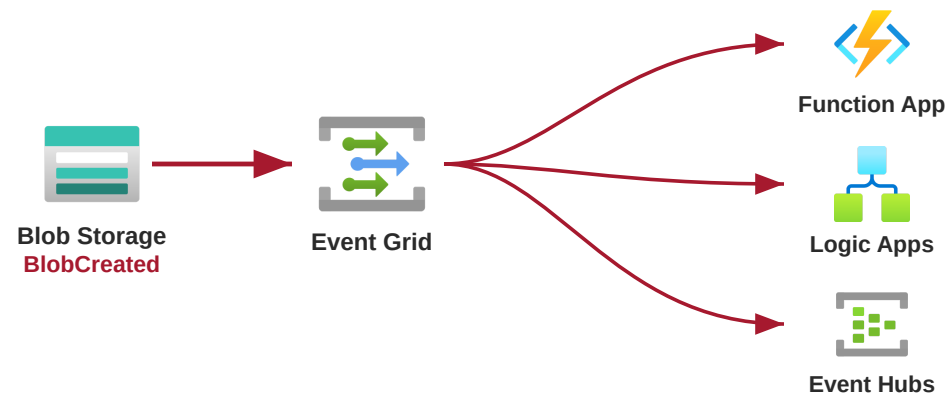
Azure separa mensajes empresariales (Service Bus) de eventos reactivos (Event Grid).

Azure Service Bus: mensajería empresarial



Colas y temas con suscripciones, filtros, sesiones para orden FIFO y cola de mensajes fallidos integrada

Azure Event Grid: eventos reactivos



Notifica hechos (se subió un archivo, se creó un recurso) y los entrega por HTTP a los suscriptores

Y en Google Cloud

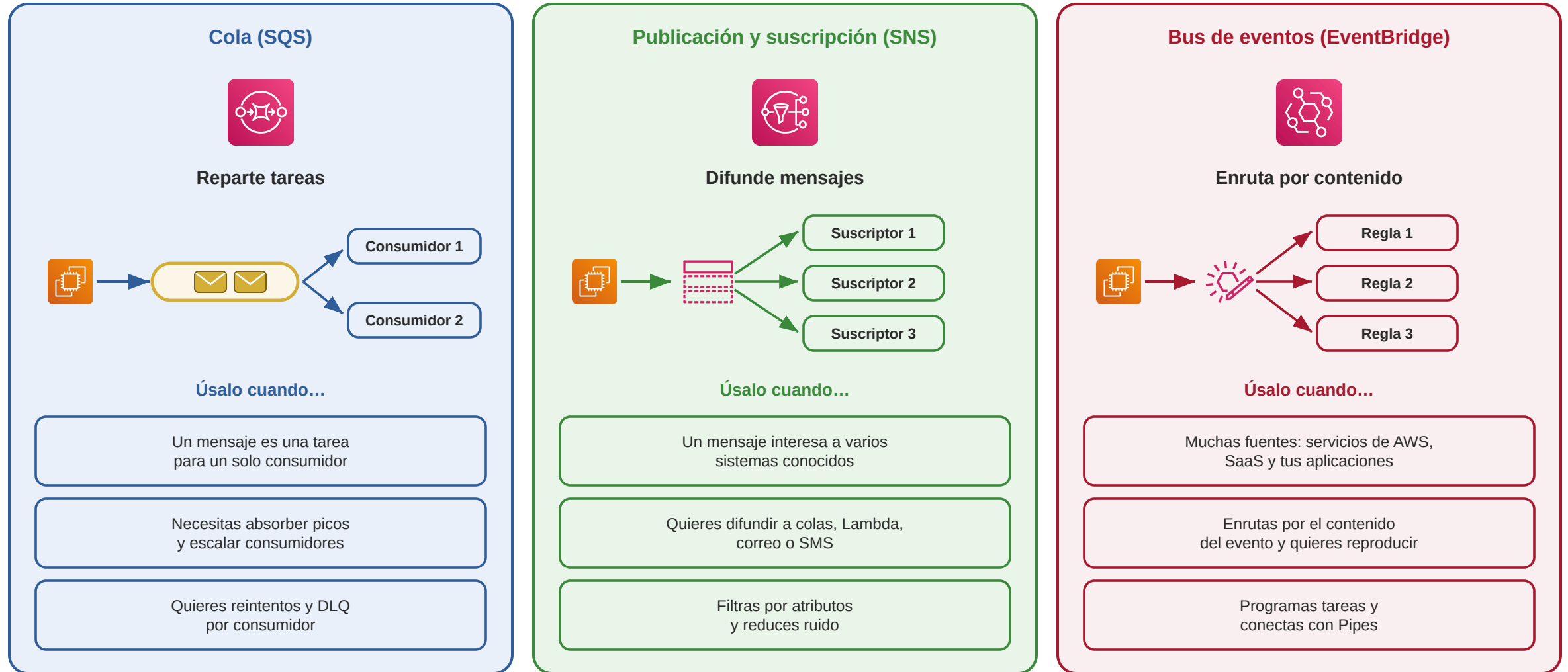


Pub/Sub: temas y suscripciones pull o push, con retención y reproducción.

Mismo modelo, otro nombre.

Colas, pub/sub y eventos: cuándo usar cada uno

No compiten: se combinan. Lo habitual es un tema o bus que difunde hacia colas que absorben.



Lo que queda de la Parte 3

Una cola reparte, un tema difunde y un bus enruta: el diseño real las combina.



Siguiente: Parte 4

Patrones de robustez y caso integrador

Idempotencia · reintentos con retroceso · orden y duplicados · streaming · Corporación Industrial del Caribe



Patrones de robustez y caso integrador

Parte 4 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Idempotencia, reintentos con retroceso, orden, streaming y el diseño final de la Corporación Industrial del Caribe.

Entrega al menos una vez: por qué llegan duplicados

Si falla la confirmación, el mensaje reaparece y el trabajo se hace dos veces.



Facturas: 2

La falla ocurrió entre hacer el trabajo y confirmar. SQS no puede saber si el trabajo se hizo: ante la duda, reentrega.

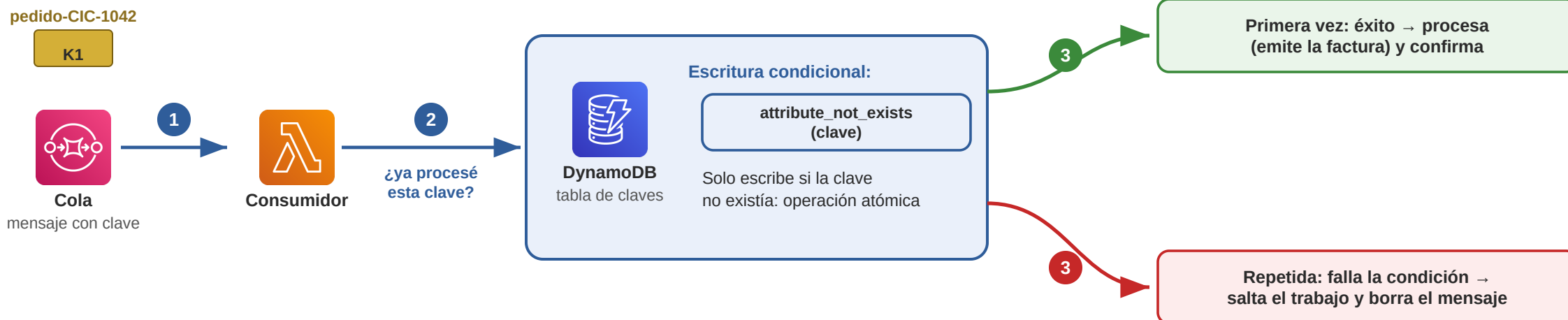
Entrega al menos una vez: la regla en sistemas distribuidos

Es mejor entregar de más que perder un mensaje. Quien consume debe poder recibir el mismo mensaje dos veces

sin cobrar dos veces ni emitir dos facturas. Esa propiedad se llama idempotencia.

Idempotencia: procesar dos veces equivale a procesar una

Una clave de idempotencia y una escritura condicional convierten dos entregas en un solo efecto.



Resultado: dos entregas, un solo efecto

Entrega 1
factura emitida

Entrega 2
reconocida y omitida

Facturas emitidas
para el pedido: 1

Detalles de diseño que cuentan:

la clave sale del negocio (el número de pedido), no del mensaje; y la tabla guarda un tiempo de expiración (TTL) para no crecer sin fin.

Si el consumidor se cae a mitad del trabajo, el registro se marca "en curso" y se reintenta; "terminado" solo al final.

Operaciones idempotentes y no idempotentes

Diseña las operaciones para asignar un estado, no para acumular un cambio.

No idempotente: aplicar dos veces cambia el resultado

saldo = saldo + 100

1 000



Inicio

1 100



Tras 1 vez

1 200



Tras 2 veces

Idempotente: aplicar dos veces da lo mismo

estado = "PAGADO"

PENDIENTE

Inicio

PAGADO

Tras 1 vez

PAGADO

Tras 2 veces

El duplicado no cambia nada

Tres formas prácticas de conseguirlo

Clave natural

Usa el número de pedido o factura como clave única.

Escritura condicional

Inserta solo si no existe; actualiza solo si el estado coincide.

Deduplicación de FIFO

MessageDeduplicationId descarta repetidos, pero solo 5 minutos.

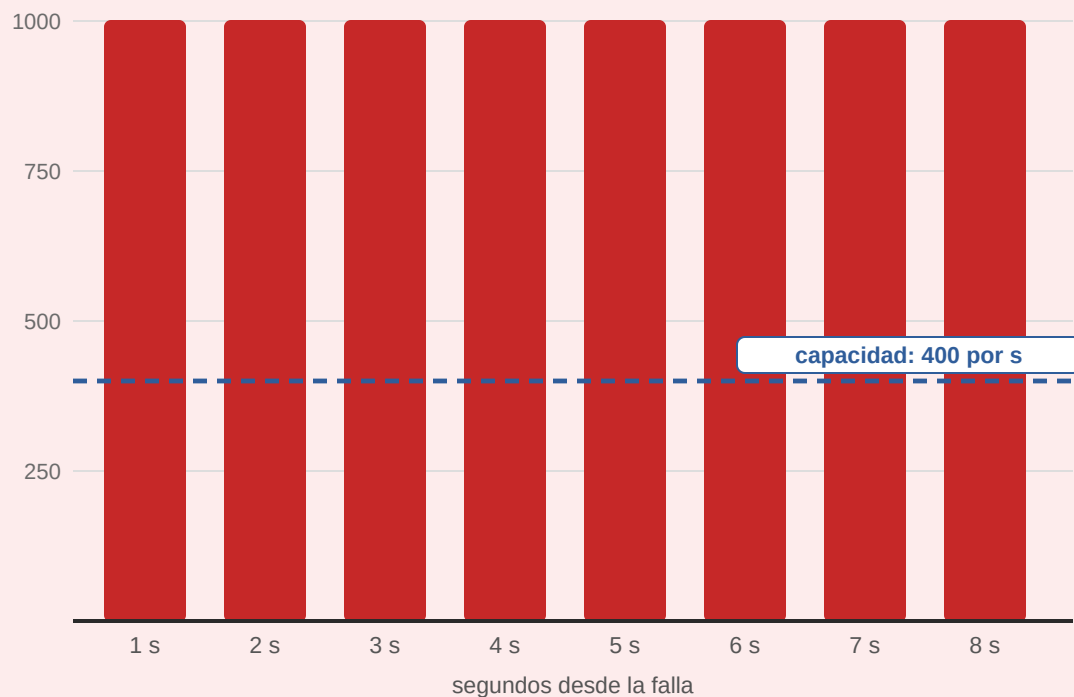
Asignar, no acumular

Prefiere "estado = X" y "upsert" a "contador + 1" o "insertar".

Reintentos: el retraso fijo sincroniza la avalancha

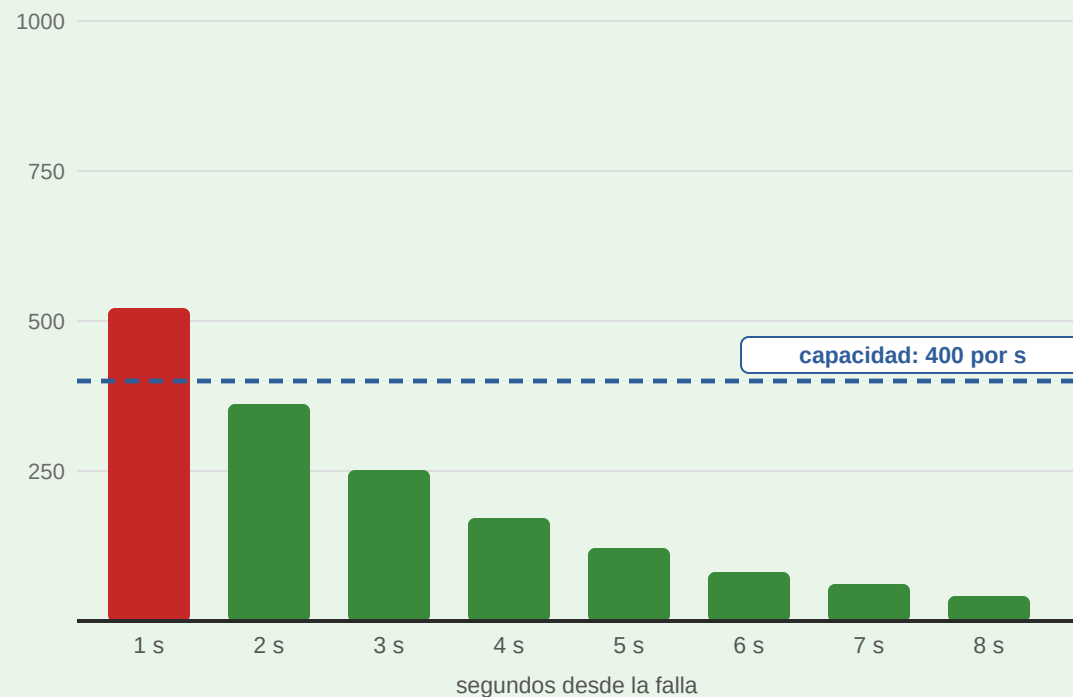
Reintentar a intervalos fijos mantiene a todos los clientes marchando al mismo paso.

Retraso fijo de 1 s: todos reintentan a la vez



Cada oleada supera la capacidad: el servicio nunca se recupera (efecto manada)

Retroceso exponencial con jitter: la carga se dispersa

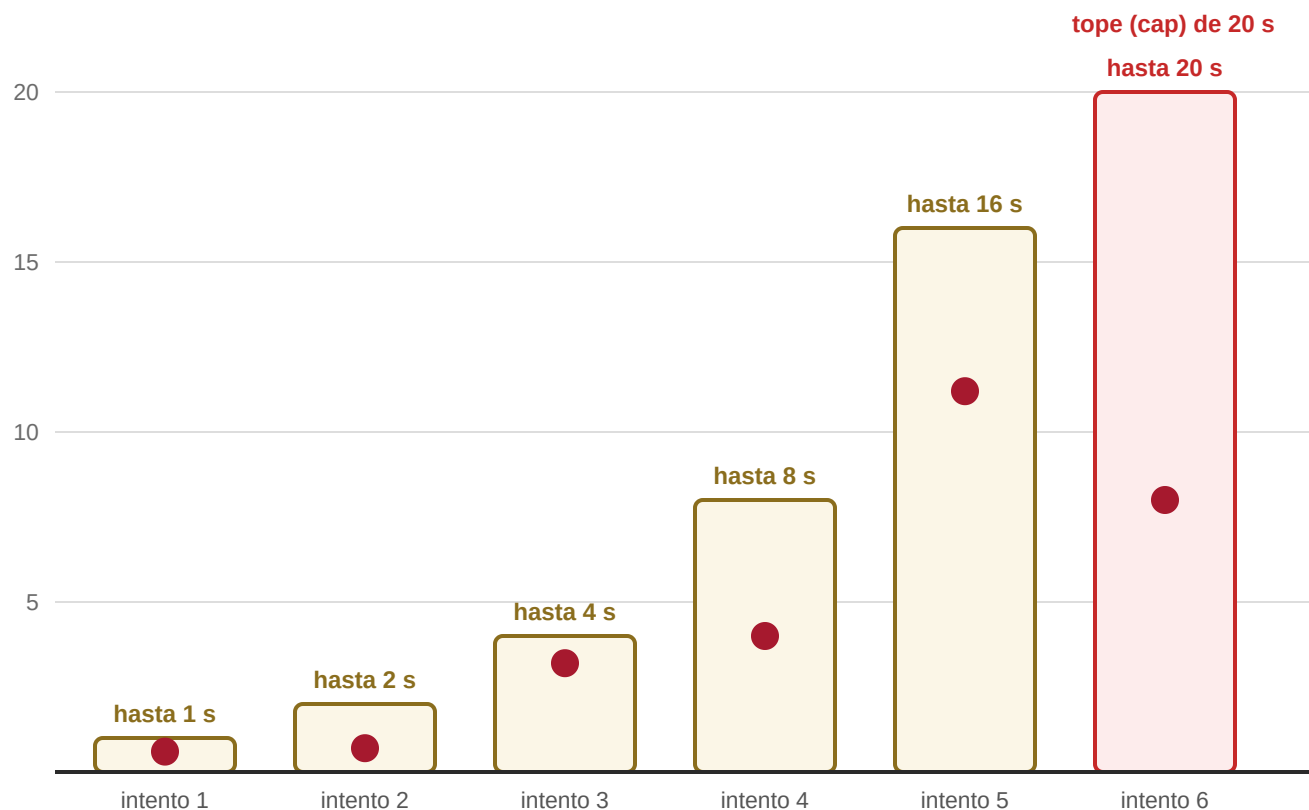


La carga queda por debajo de la capacidad desde el segundo 2 y el servicio se recupera

Retroceso exponencial con jitter

Esperar el doble en cada intento, con un tiempo aleatorio, protege al servicio que se recupera.

Espera máxima antes del siguiente intento (segundos)



- espera real elegida al azar entre 0 y el máximo de cada barra

Valores de ejemplo: base 1 s, factor 2, tope 20 s

Receta de reintento seguro

$\text{espera} = \text{aleatorio}(0, \min(\text{tope}, \text{base} \times 2^{\text{intento}}))$

- 1 Exponencial**
cada intento espera el doble
- 2 Jitter**
valor aleatorio: rompe la sincronía
- 3 Tope**
evita esperas de minutos
- 4 Máximo de intentos**
después, a la DLQ

Los SDK de AWS ya reintentan con retroceso exponencial y jitter; tú decides el máximo y qué errores se reintentan

Orden y duplicados: qué garantiza cada servicio

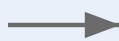
El orden casi siempre es por grupo o por partición, nunca global; los duplicados hay que asumirlos.



SQS estándar

alto rendimiento

Entra



Sale



Orden: aproximado

Diseña consumidores idempotentes

Duplicados: posibles



SQS FIFO

orden estricto

Entra



Sale



Orden: por grupo

Un grupo por entidad: producto o pedido

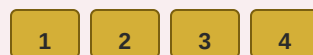
Duplicados: no (5 min)



Kinesis o Kafka

registro ordenado

Entra



Sale



Orden: por fragmento

Misma clave de partición, mismo fragmento

Duplicados: posibles



EventBridge y SNS estándar

enrutamiento

Entra



Sale



Orden: no garantizado

No dependas del orden; incluye marca de tiempo

Duplicados: posibles

Mensajería frente a streaming: dos modelos

La cola entrega y olvida; el registro de streaming conserva y permite releer.

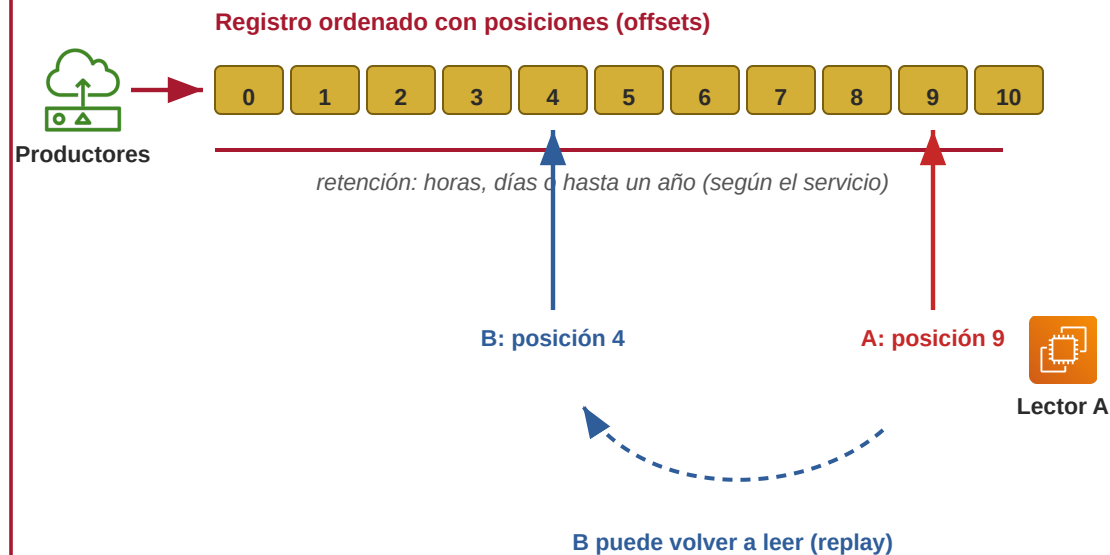
Mensajería (cola): el mensaje se consume y desaparece



- 1 Cada mensaje se entrega a un consumidor
- 2 Al confirmar, se borra de la cola
- 3 No se puede releer lo ya procesado

Ideal para tareas: pedidos, correos, facturas.
El trabajo se reparte y desaparece al terminar.

Streaming (registro): los datos se conservan y se releen



Varios lectores independientes, cada uno con su posición

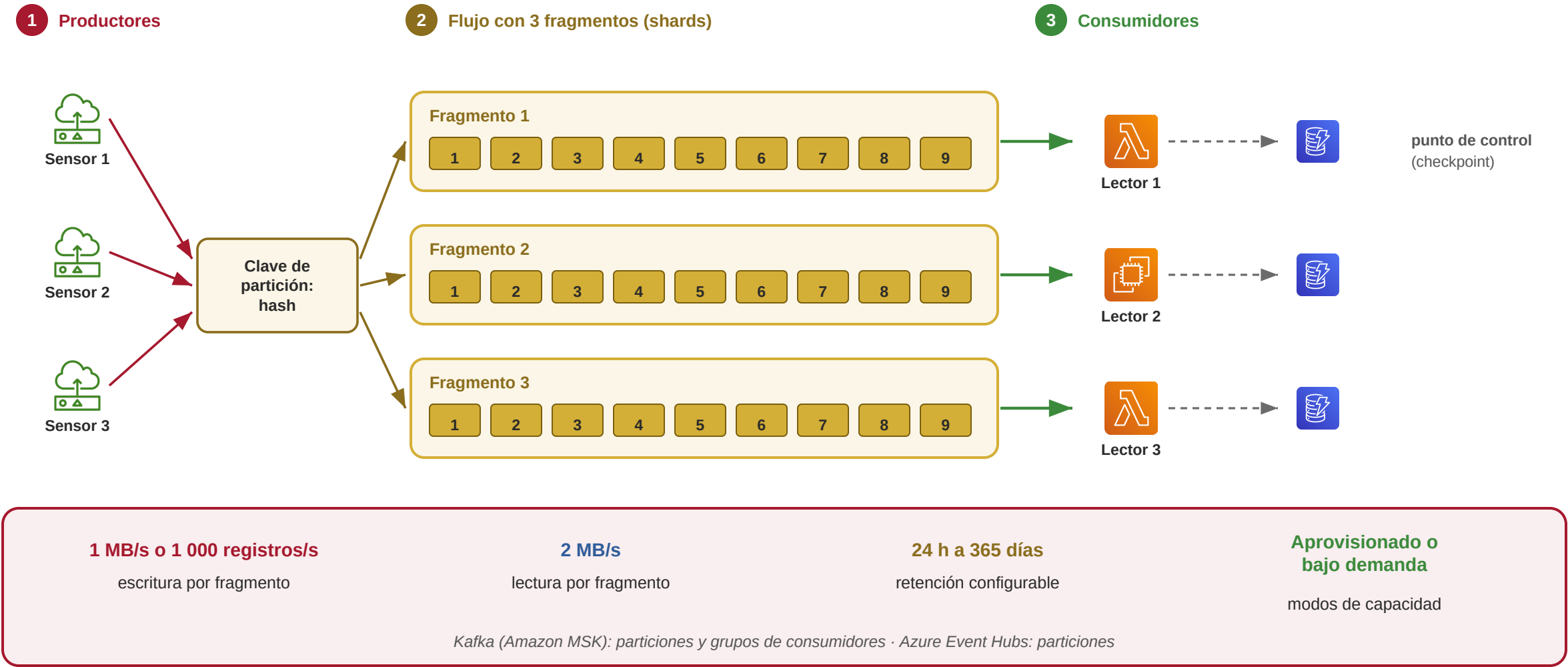
Los registros no se borran al leerlos

Puedes releer el pasado y reconstruir resultados

Ideal para flujos continuos: sensores, clics, registros.

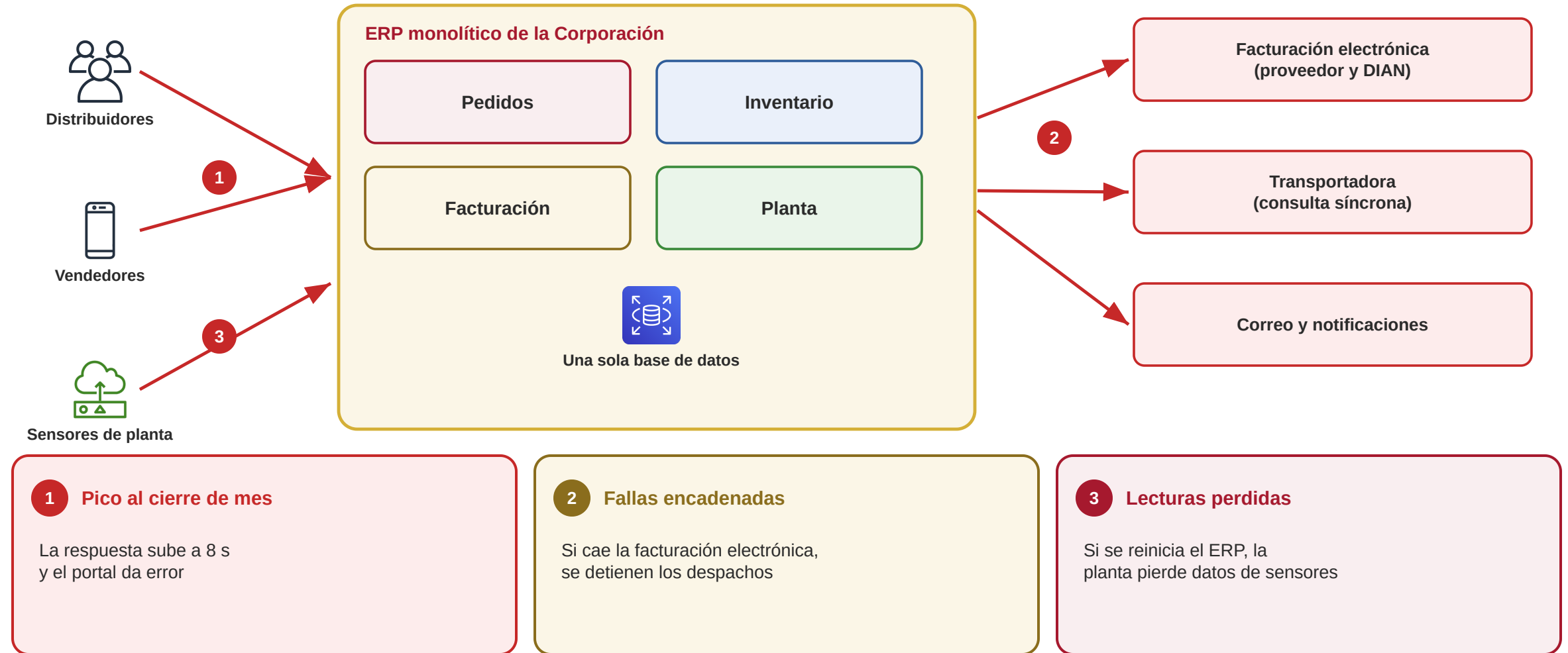
Kinesis y Kafka: fragmentos, particiones y lectores

La clave de partición decide el fragmento; dentro de cada fragmento el orden se conserva.



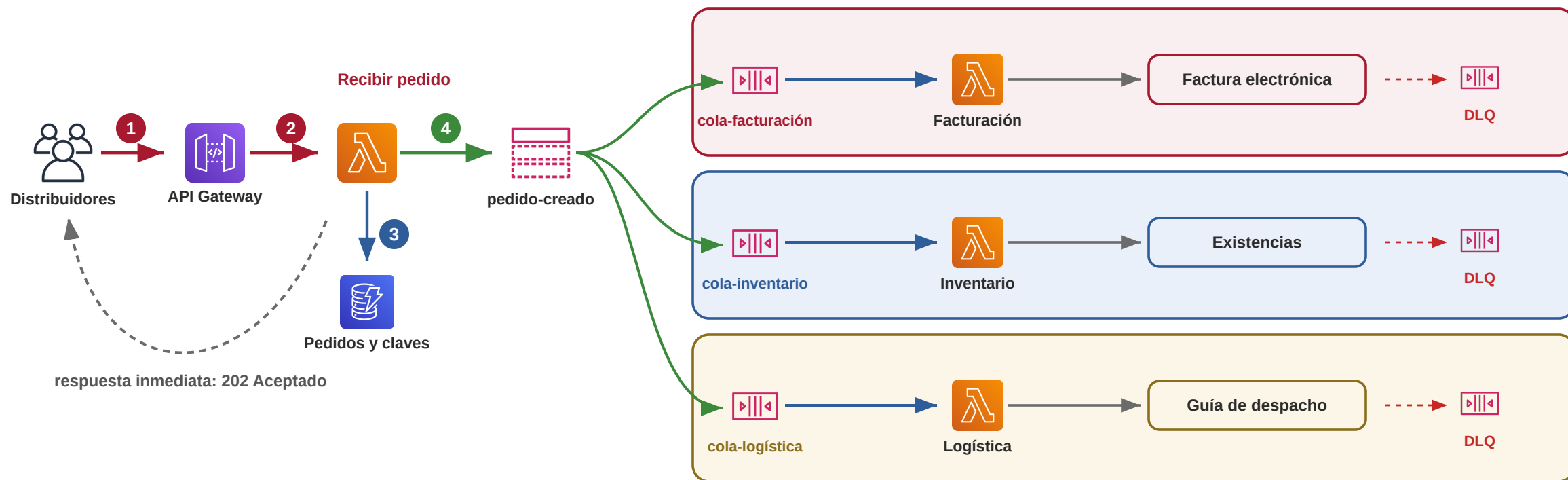
Caso: la Corporación Industrial del Caribe hoy

Todo habla con todo de forma síncrona dentro de un ERP: cualquier pico o falla se propaga.



Caso: pedidos desacoplados con SNS y SQS

El portal confirma de inmediato; facturación, inventario y logística trabajan cada uno a su ritmo.



Reintentos con retroceso exponencial y jitter

Consumidores idempotentes por número de pedido

DLQ con alarma de CloudWatch en cada cola

Escala por profundidad de la cola

Resultado: el distribuidor recibe la confirmación en menos de un segundo, aunque Facturación esté lenta o caída.

CASO

Caso: la planta, sensores y alarmas

Streaming para el flujo continuo de lecturas; un evento solo cuando hay algo que atender.



Por qué streaming

- 1 Miles de lecturas por segundo, de forma continua
- 2 El orden por sensor es importante
- 3 Releer sirve para analizar una falla pasada

Dos canales para dos necesidades

Flujo continuo y volumen alto: streaming (Kinesis) y almacenamiento histórico en S3

Hecho puntual que exige acción: evento (EventBridge) y aviso con SNS y una cola

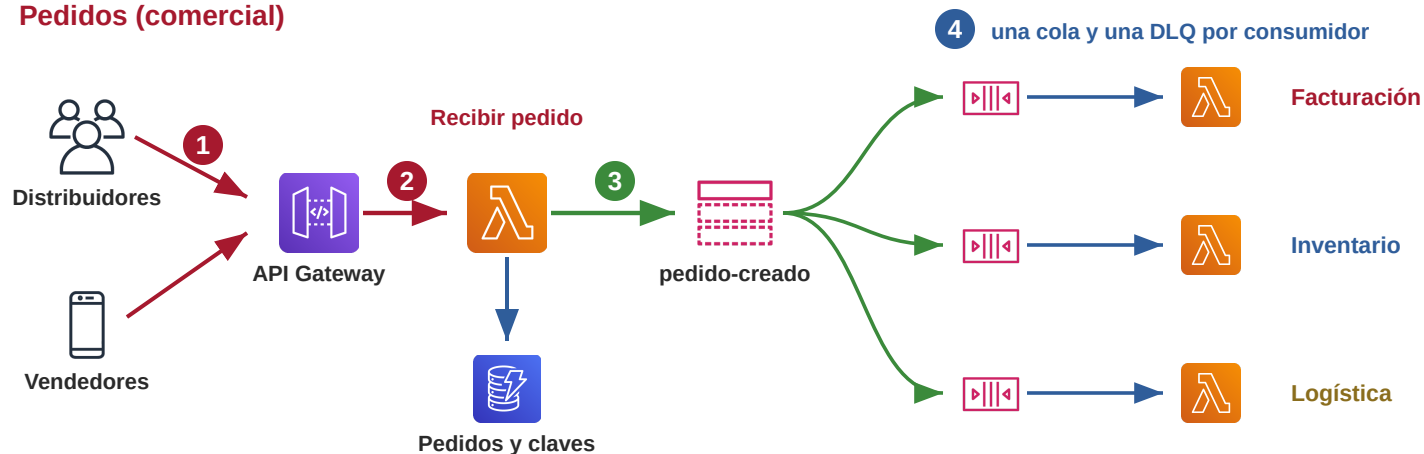
Las lecturas normales nunca saturan a los consumidores de alarmas: solo el evento excepcional atraviesa el bus.

Umbral y volúmenes ilustrativos.

Arquitectura desacoplada final de la Corporación

Dos flujos, pedidos y planta, con colas, temas, streaming y eventos: cada pieza falla y escala sola.

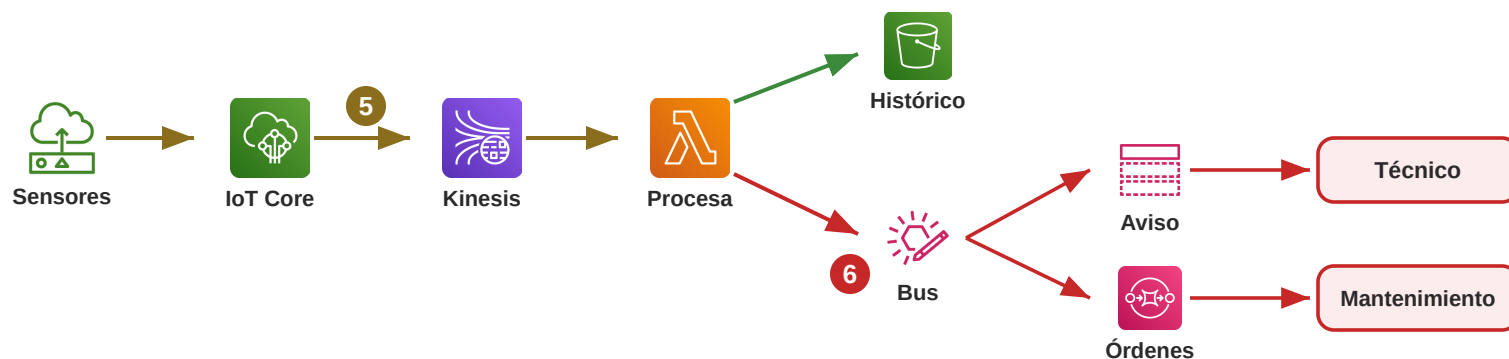
Pedidos (comercial)



Garantías de la parte comercial

- Respuesta inmediata al distribuidor
- Idempotencia por número de pedido
- Reintentos con retroceso y jitter
- Alarma de CloudWatch en cada DLQ
- Escala por profundidad de cola

Planta (operación)



Observabilidad y seguridad comunes

CloudWatch
alarmas y métricas

IAM
mínimo privilegio

Registros
trazas con id de pedido

Todo en una VPC de dos zonas de disponibilidad;
los servicios administrados replican entre zonas.

Semana 13 en cuatro ideas y adelanto de la Semana 14

Desacoplar con mensajes da resiliencia; hacerlo bien exige idempotencia, reintentos y vigilancia.

1 Desacopla en el tiempo



Un intermediario entre productor y consumidor da resiliencia.

Resiliencia

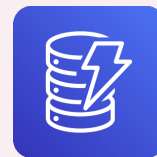
2 Cola, tema o bus



Reparte tareas, difunde noticias o enruta eventos por contenido.

Elige con criterio

3 Asume duplicados



Idempotencia, reintentos con retroceso y jitter, y DLQ.

Robustez

4 Mensaje o flujo



La cola borra al consumir; el streaming conserva y relee.

Dos modelos

Próxima semana: Semana 14

Serverless y microservicios

Funciones como servicio (FaaS) · gestión de APIs · Laboratorio AWS: arquitectura serverless (10 %)

